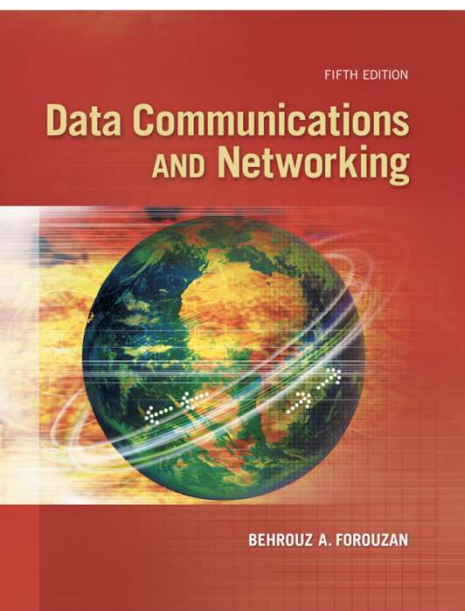


Computer Networks

Transport Layer Protocols



AGENDA

Transport Layer Protocols

- Simple protocol

- Stop and wait protocol

- Go back N protocol

- Selective Repeat protocol



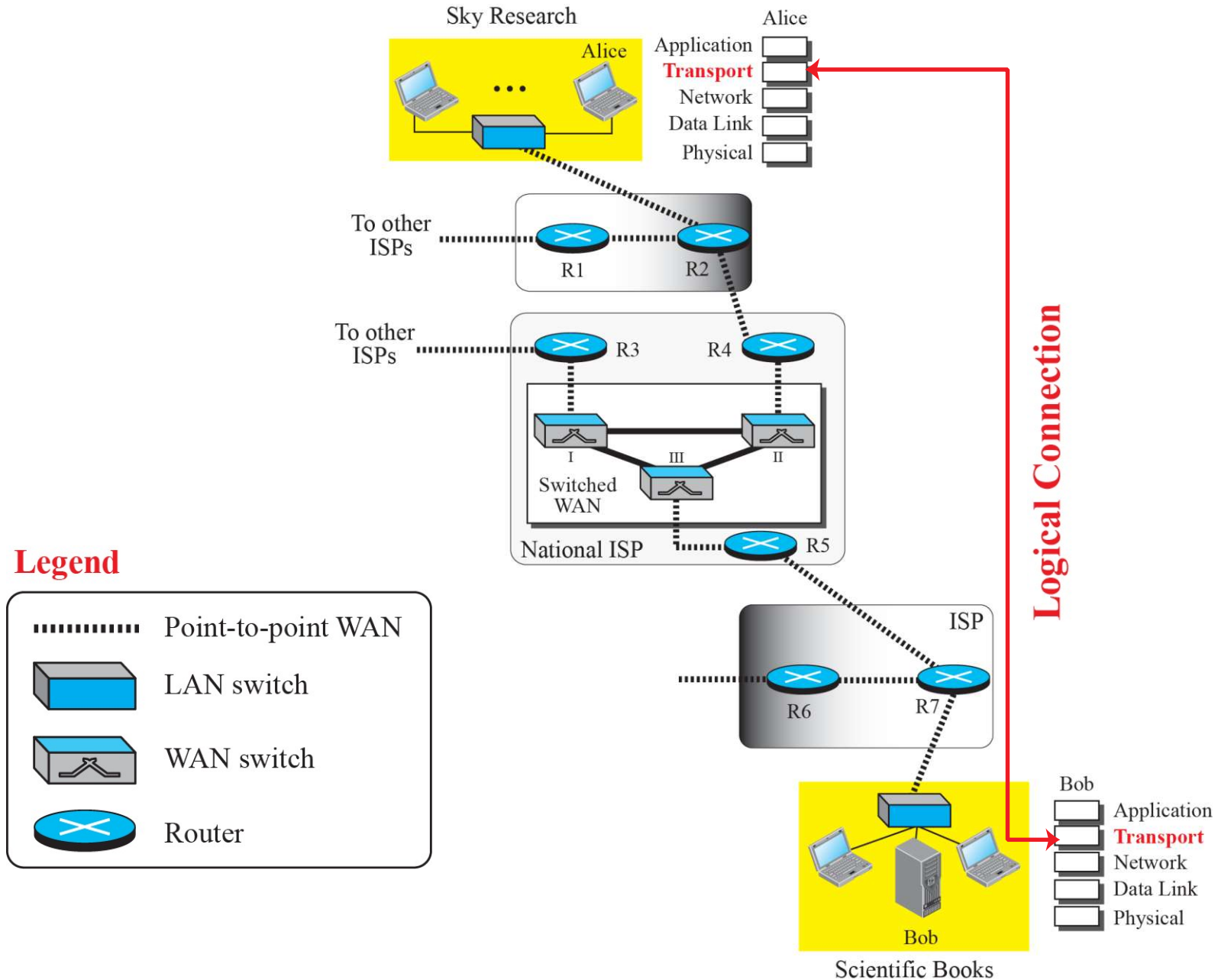
INTRODUCTION

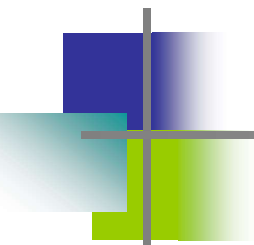
The transport layer is located between the application layer and the network layer.

It provides a process-to-process communication between two application layers, one at the local host and the other at the remote host.

Communication is provided using a logical connection.

Logical connection at the transport layer



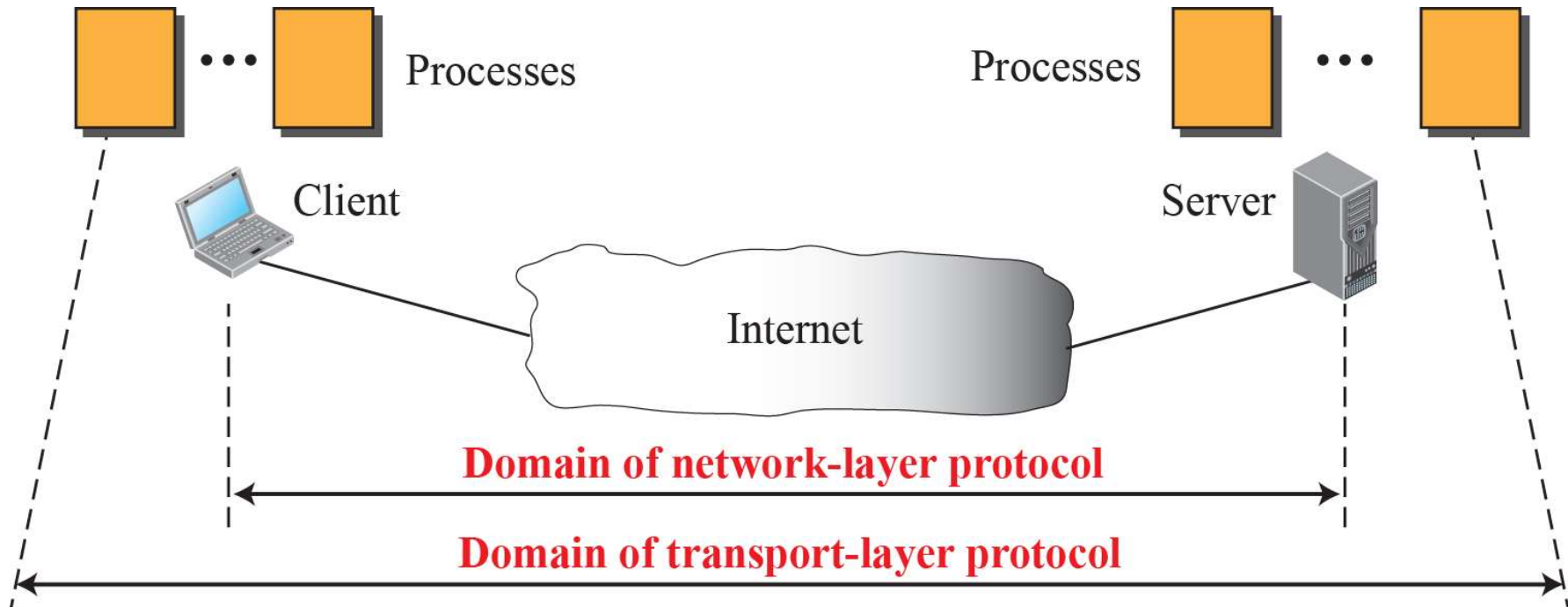


Transport-Layer Services

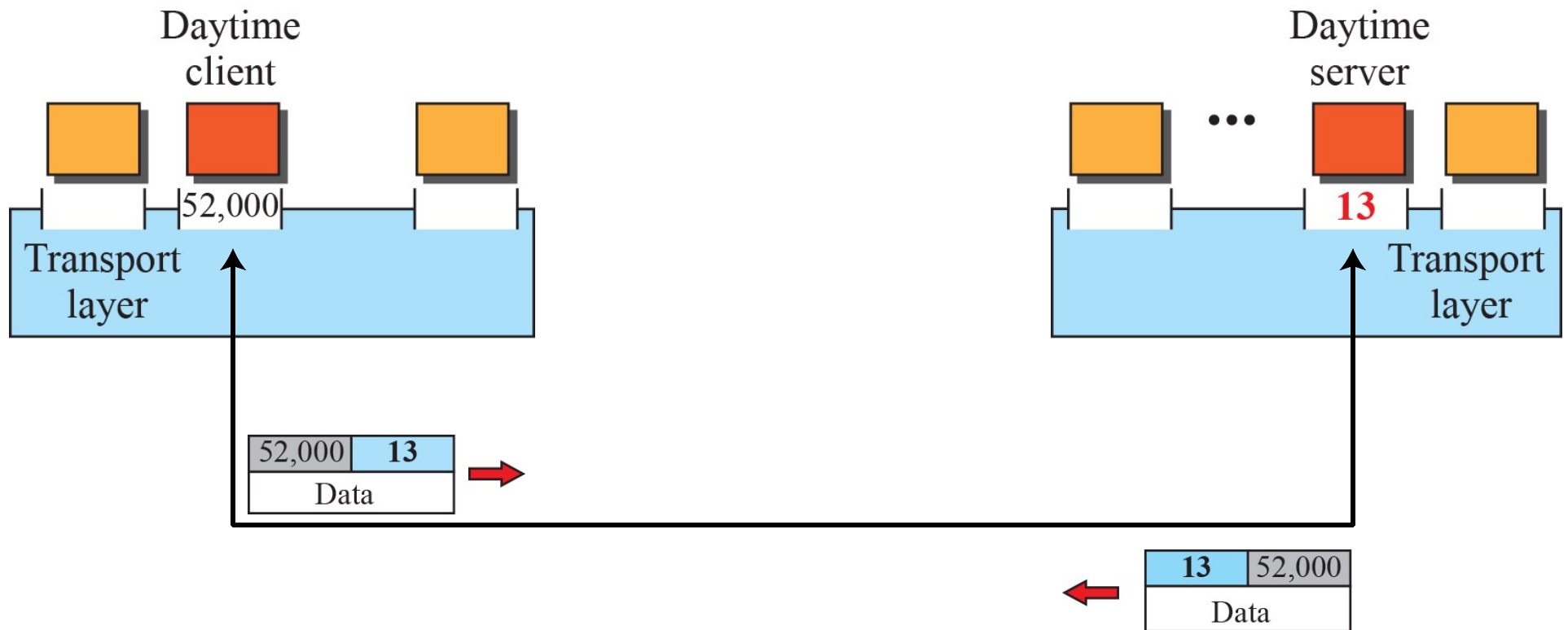
Transport layer is located between the network layer and the application layer.

The transport layer is responsible for providing services to the application layer; it receives services from the network layer.

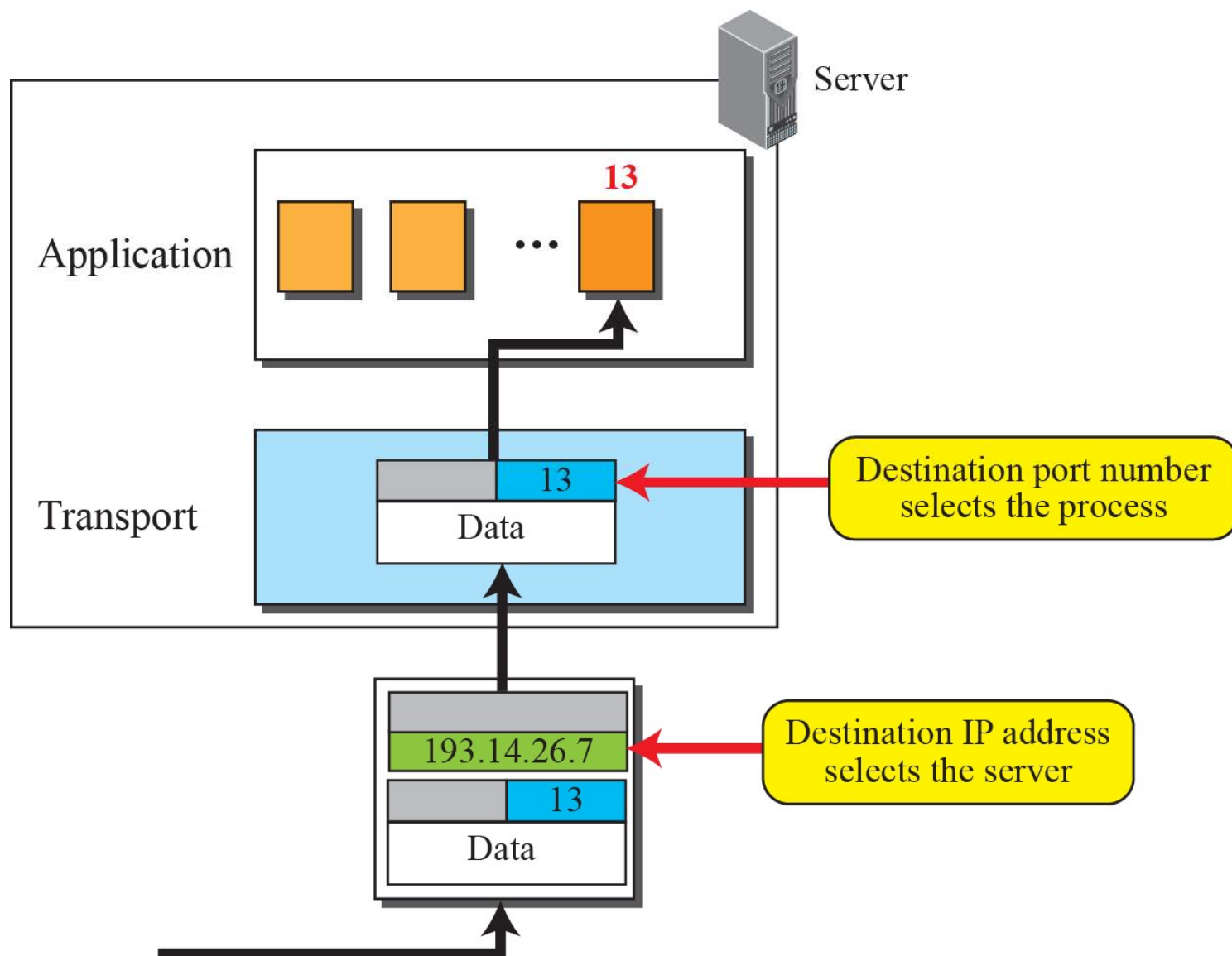
Network layer versus transport layer



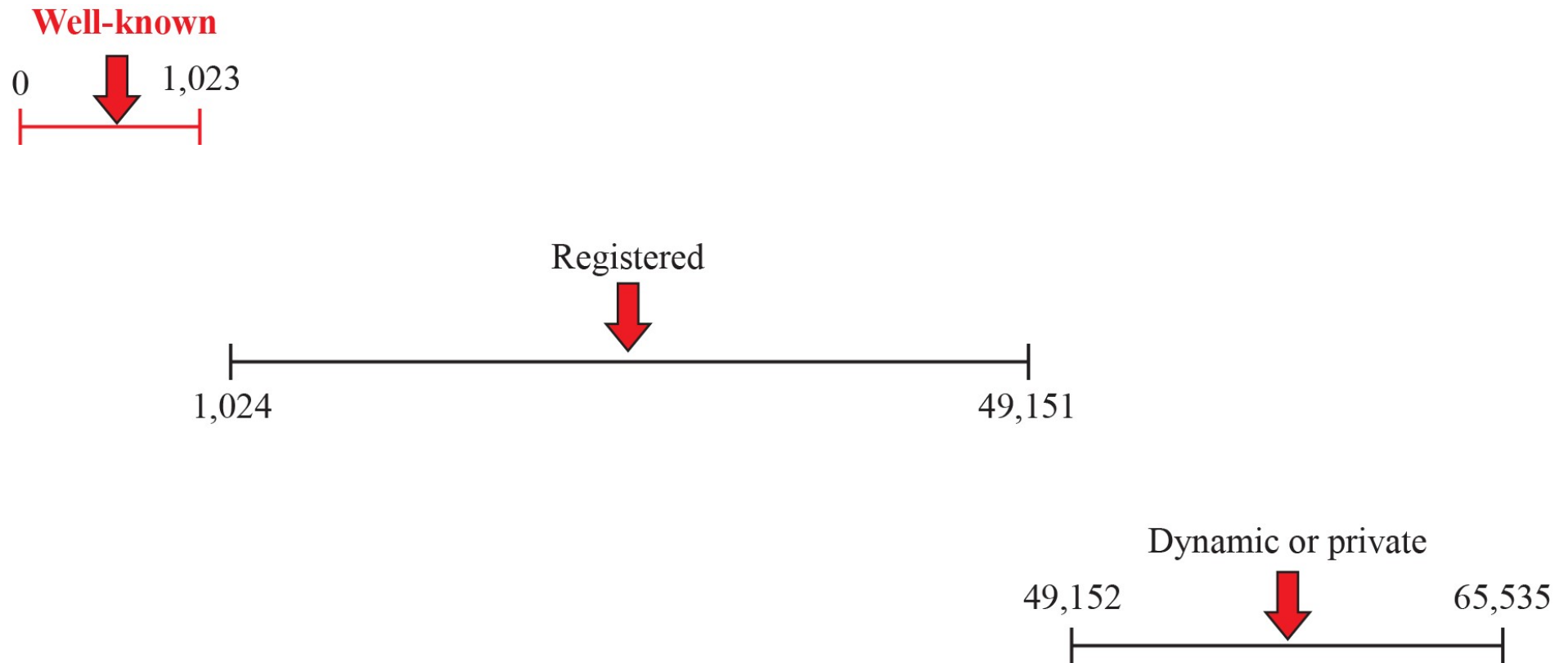
Port numbers



IP addresses versus port numbers



ICANN ranges



Example

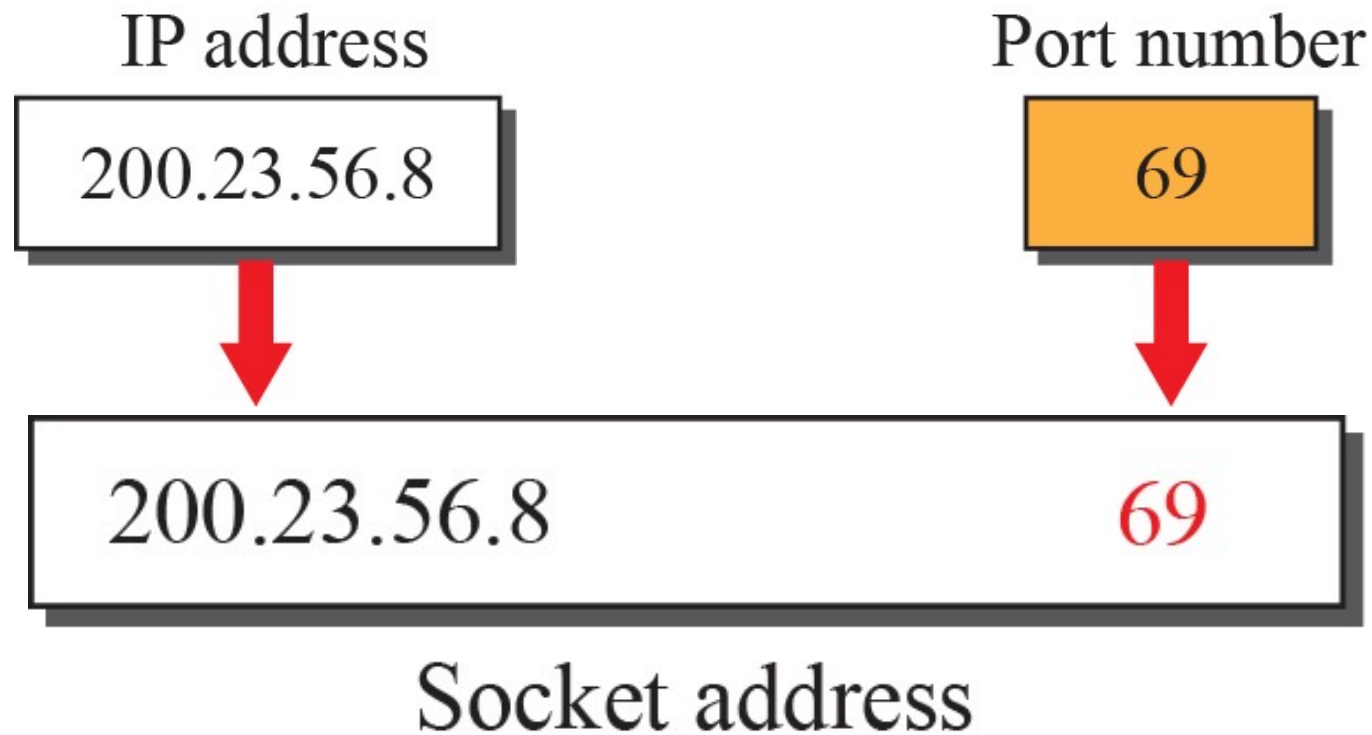
In UNIX, the well-known ports are stored in a file called `/etc/services`. We can use the `grep` utility to extract the line corresponding to the desired application.

```
$grep tftp/etc/services  
tftp 69/tcp  
tftp 69/udp
```

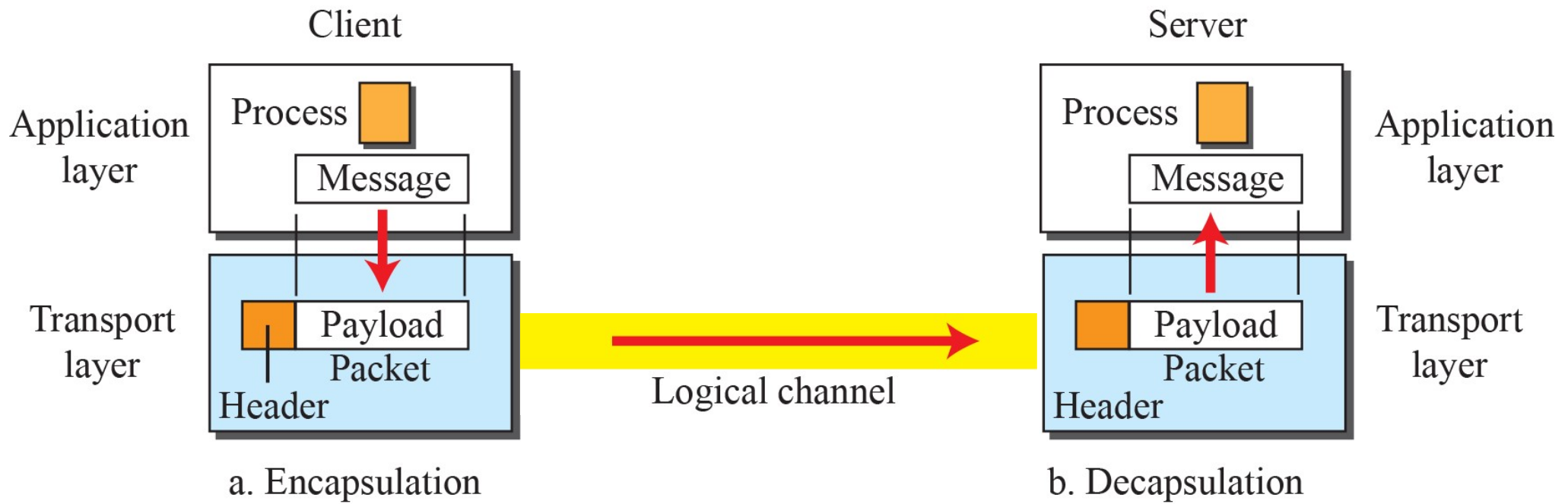
SNMP (see Chapter 27) uses two port numbers (161 and 162), each for a different purpose.

```
$grep snmp/etc/services  
snmp161/tcp#Simple Net Mgmt Proto  
snmp161/udp#Simple Net Mgmt Proto  
snmptrap162/udp#Traps for SNMP
```

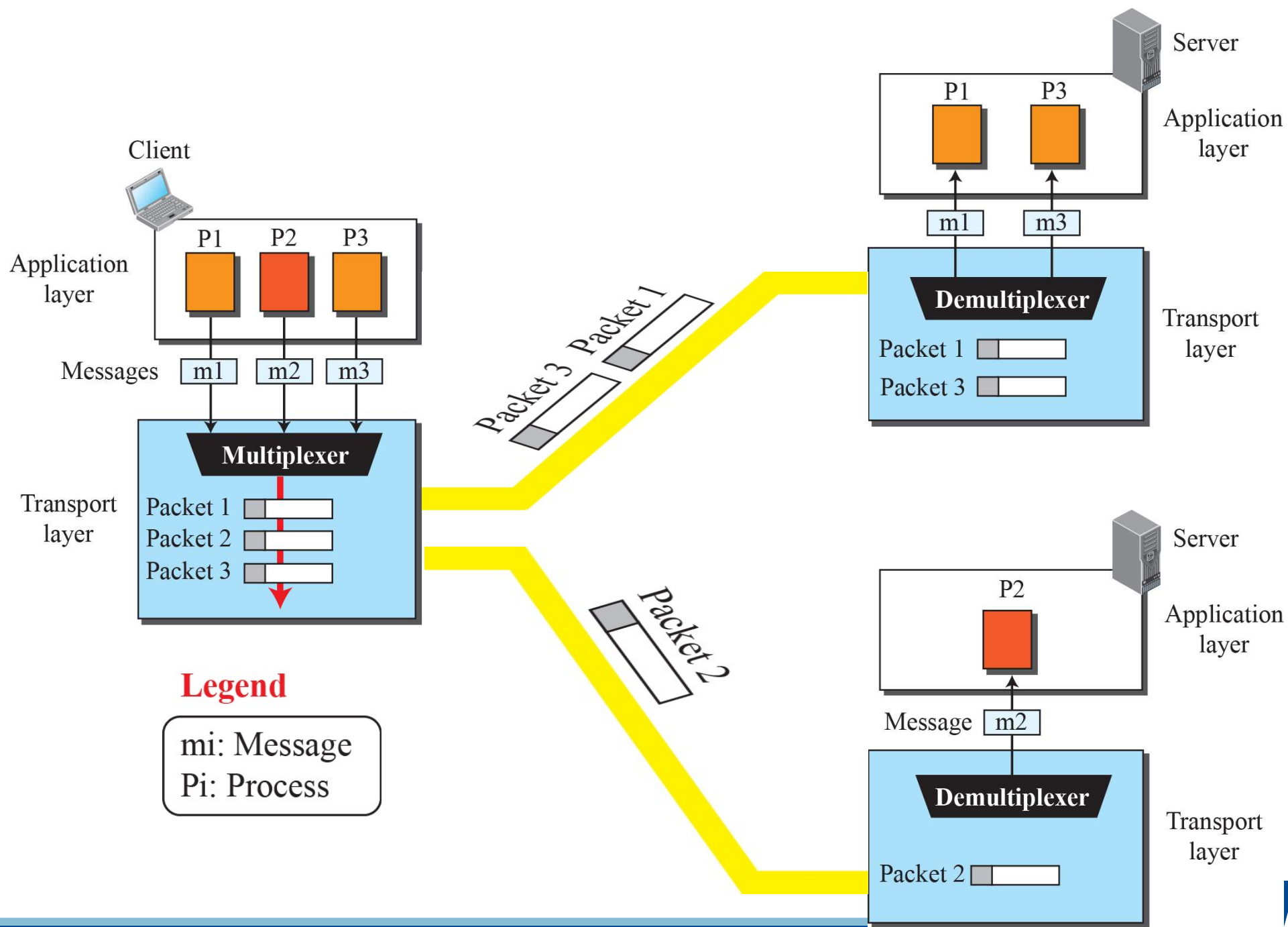
Socket address



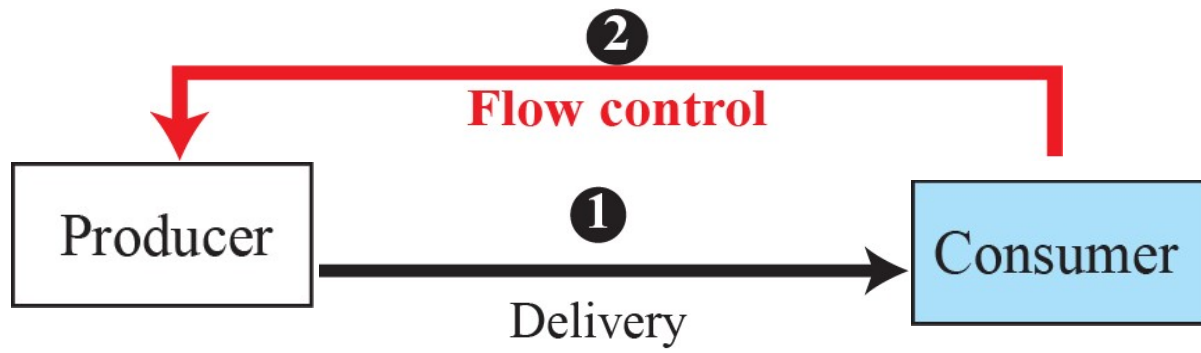
Encapsulation and decapsulation



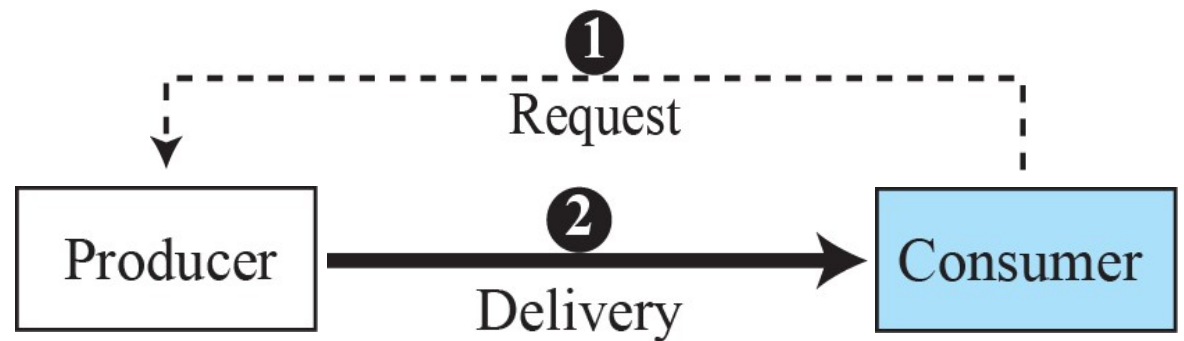
Multiplexing and demultiplexing



Pushing or pulling

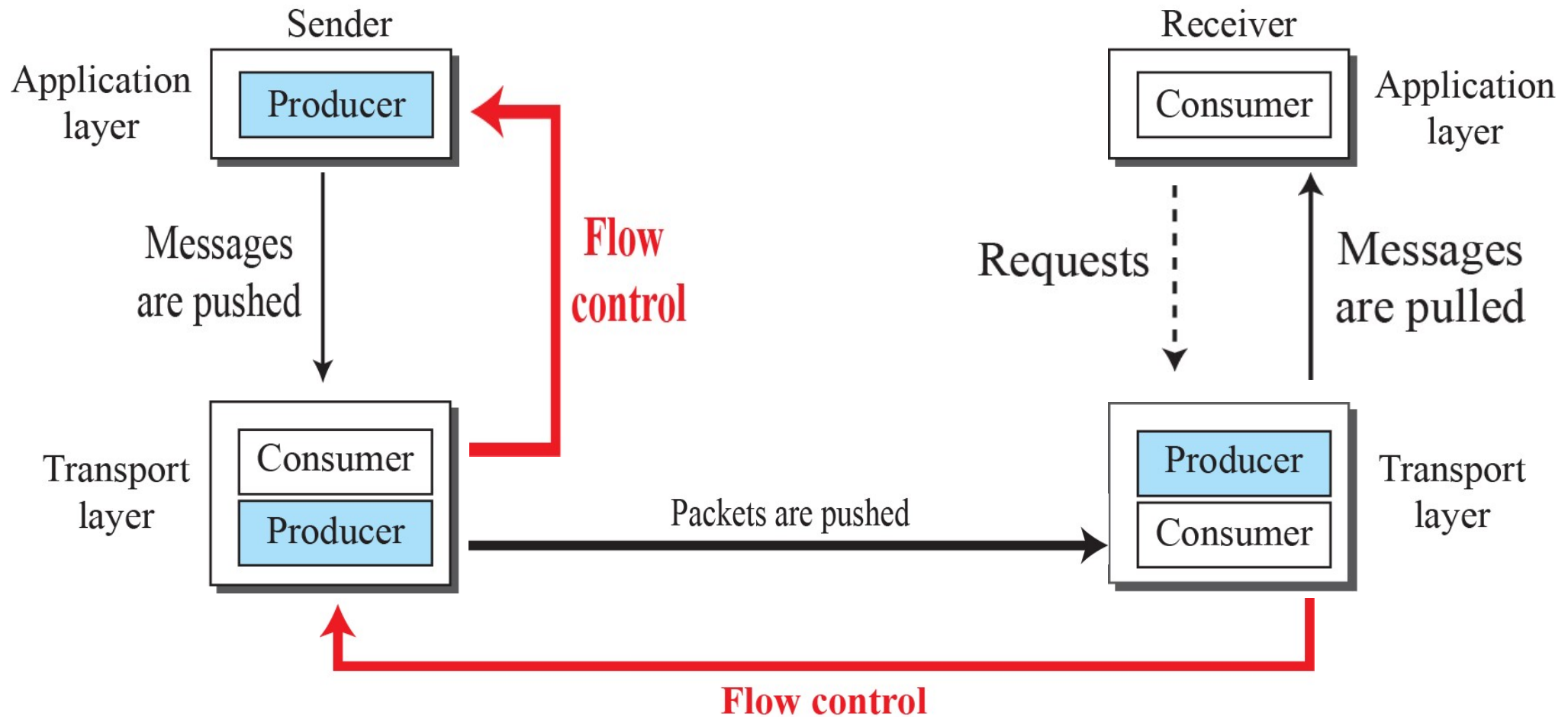


a. Pushing



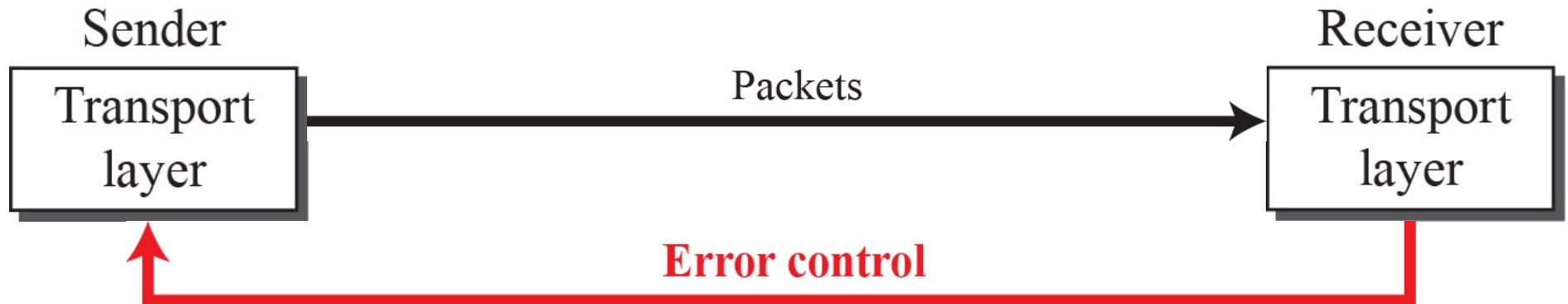
b. Pulling

Flow control at the transport layer

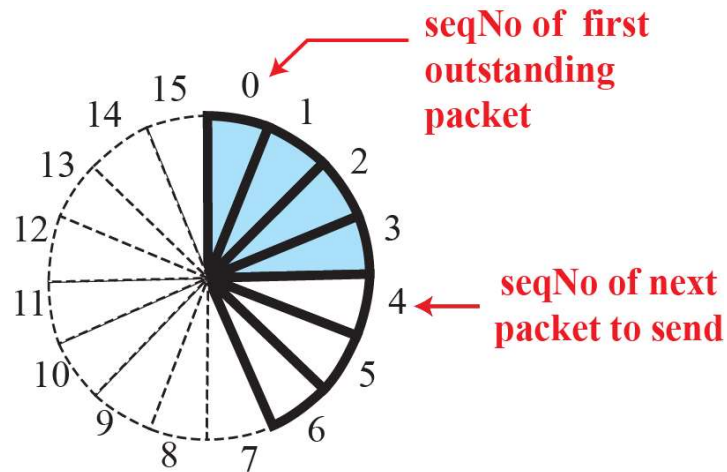


Error control at the transport layer

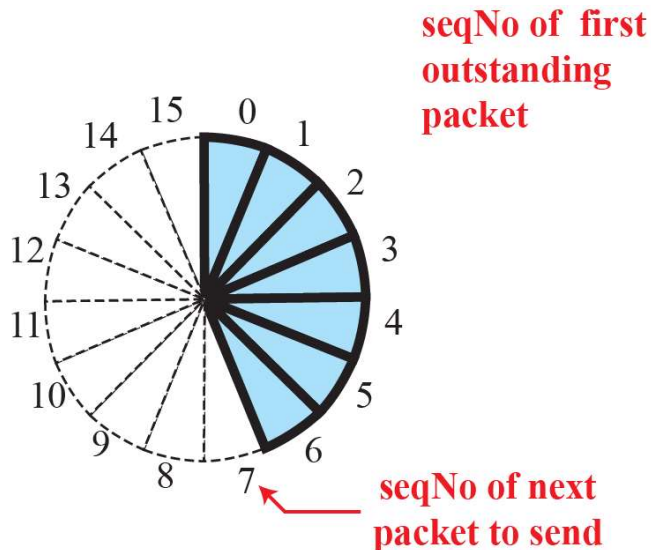
1. Detecting and discarding corrupted packets.
2. Keeping track of lost and discarded packets and resending them.
3. Recognizing duplicate packets and discarding them.
4. Buffering out-of-order packets until the missing packets arrive.



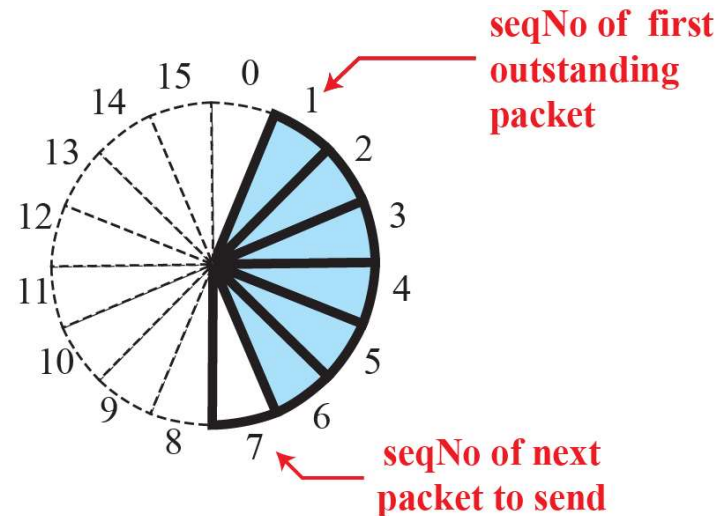
Sliding window in circular format



a. Four packets have been sent.



c. Seven packets have been sent;
window is full.

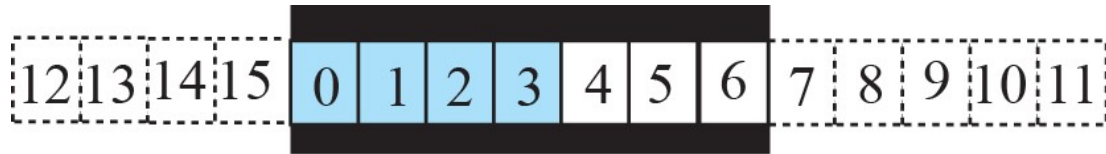


d. Packet 0 has been acknowledged;
window slides.

If the header of the packet allows m bits for the sequence number, the sequence numbers range from 0 to $2^m - 1$. Eg: $m=4$, Seq.No: 0 to 15

Sequence numbers are modulo 2^m

Sliding window in linear format



a. Four packets have been sent.



b. Five packets have been sent.

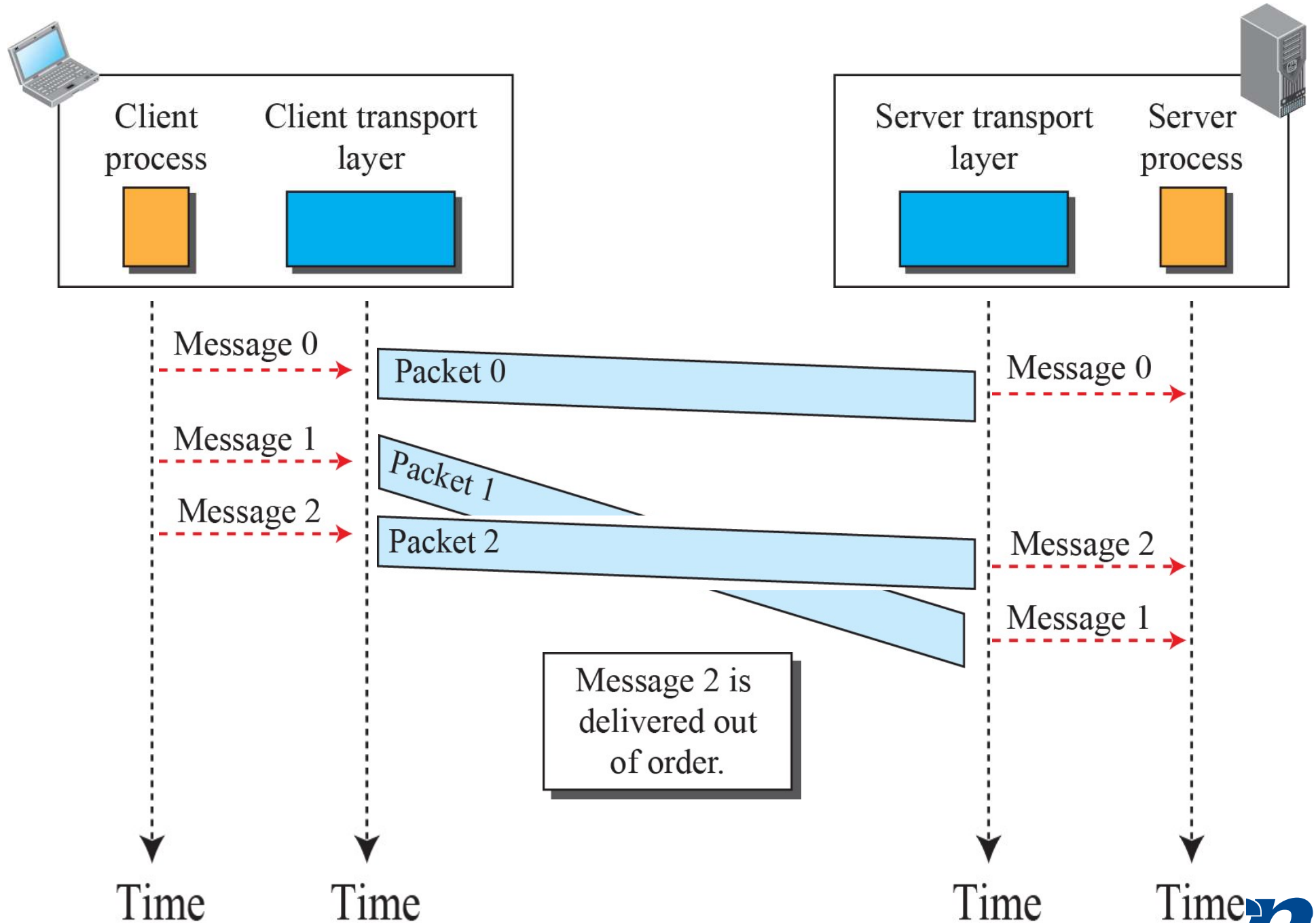


c. Seven packets have been sent;
window is full.

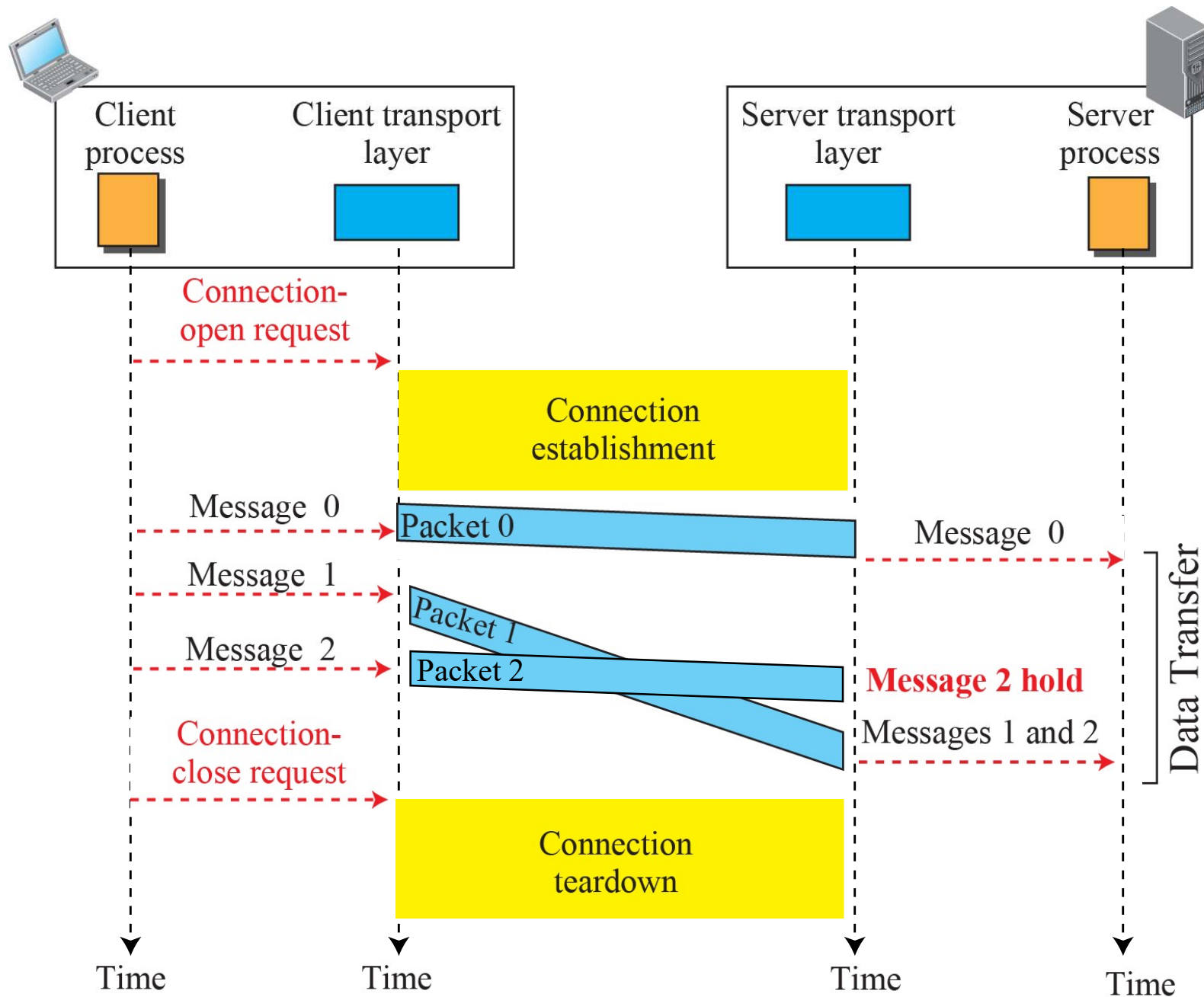


d. Packet 0 has been acknowledged;
window slides.

Connectionless service

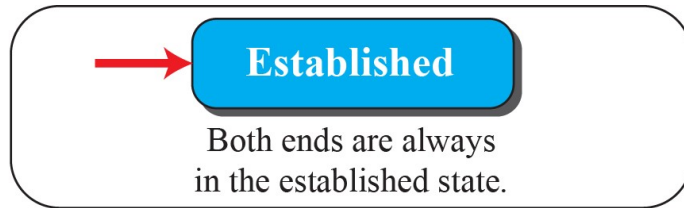


Connection-oriented service



Connectionless and connection-oriented service represented as FSMs

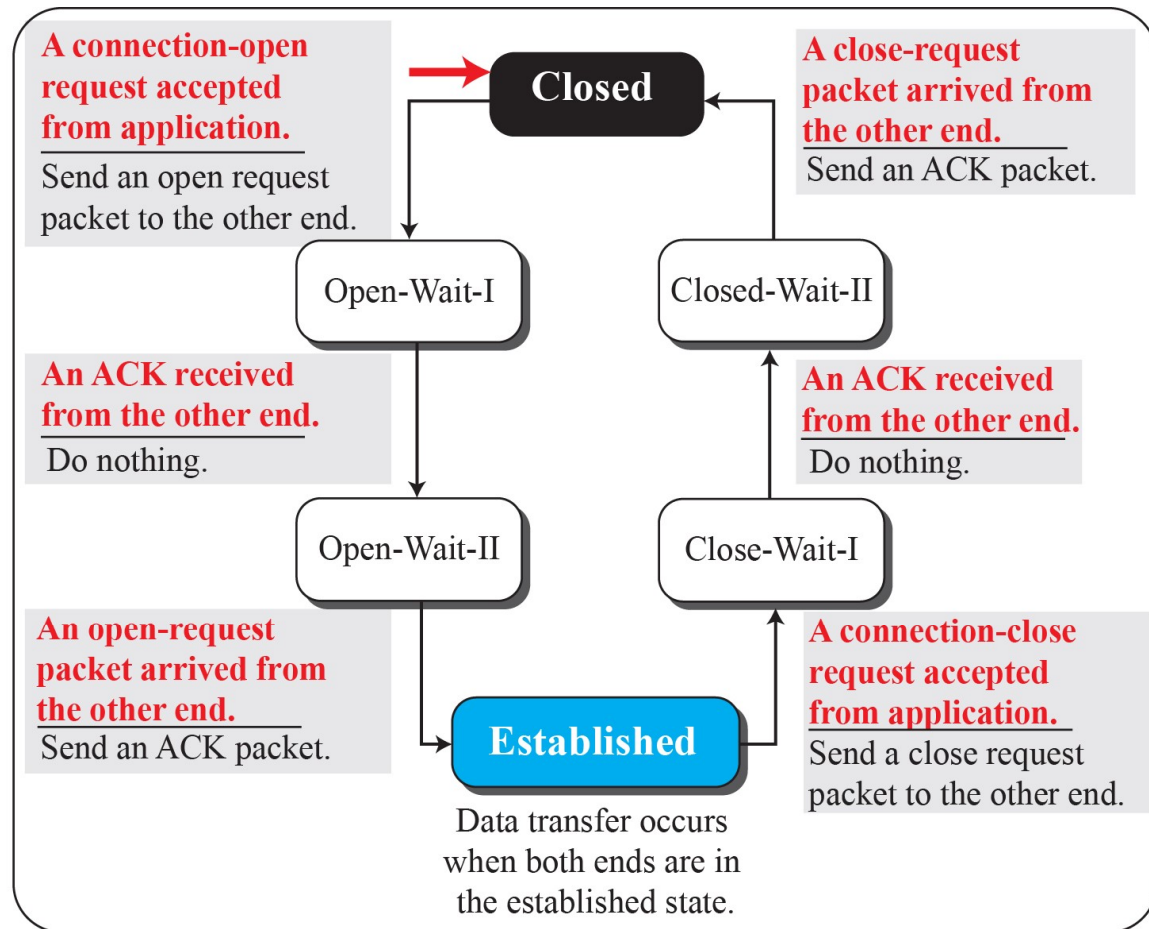
FSM for connectionless transport layer



Note:

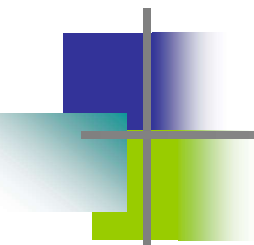
The colored arrow shows the starting state.

FSM for connection-oriented transport layer



TRANSPORT-LAYER PROTOCOLS

The TCP/IP protocol uses a transport-layer protocol that is either a modification or a combination of some of these protocols.



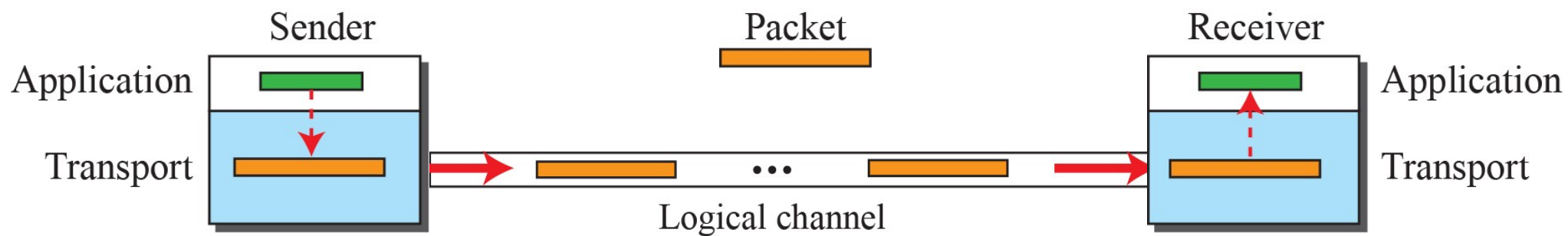
Simple Protocol

Our first protocol is a simple connectionless protocol with neither flow nor error control.

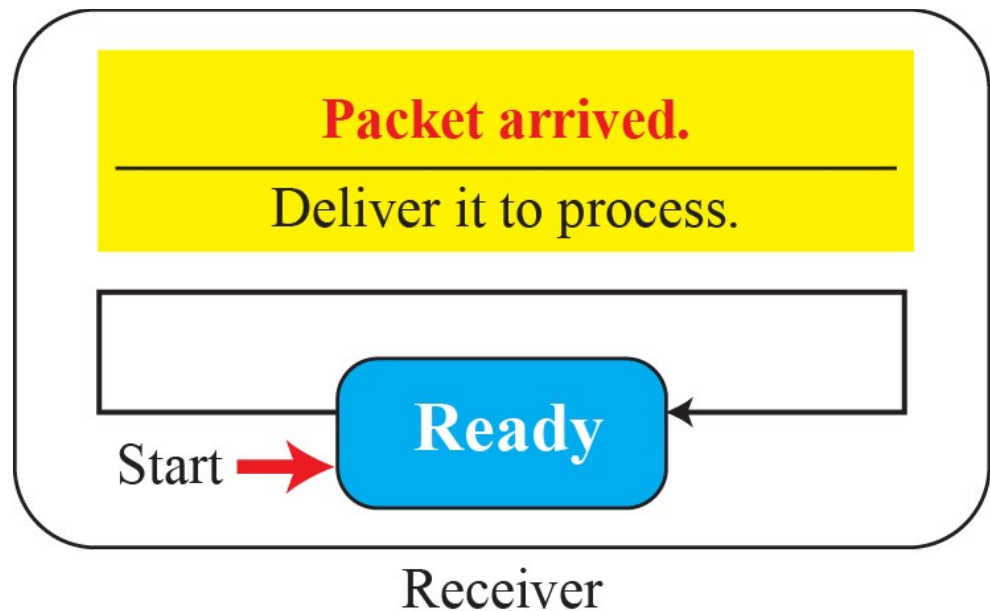
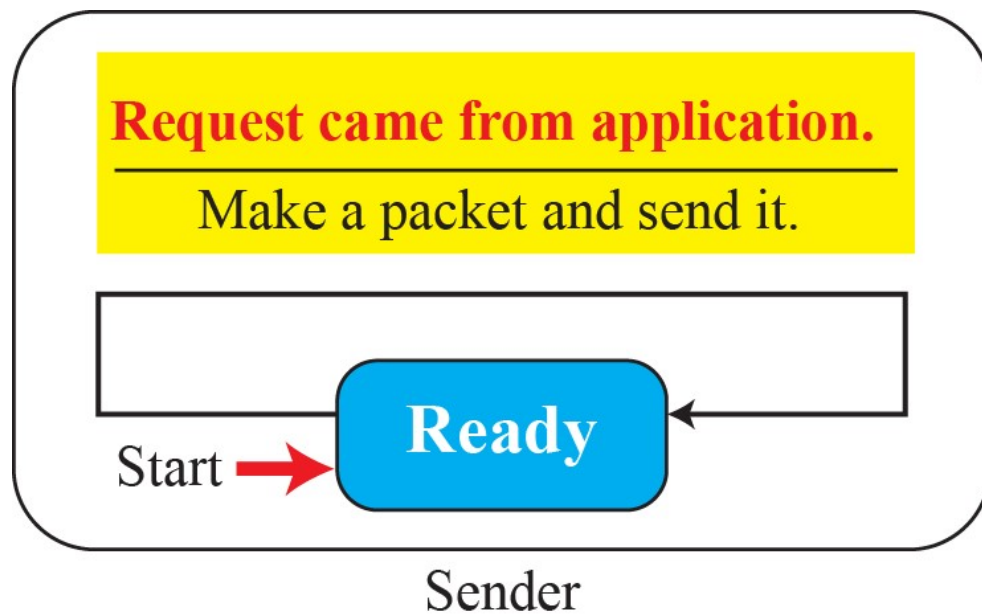
We assume that the receiver can immediately handle any packet it receives.

In other words, the receiver can never be overwhelmed with incoming packets.

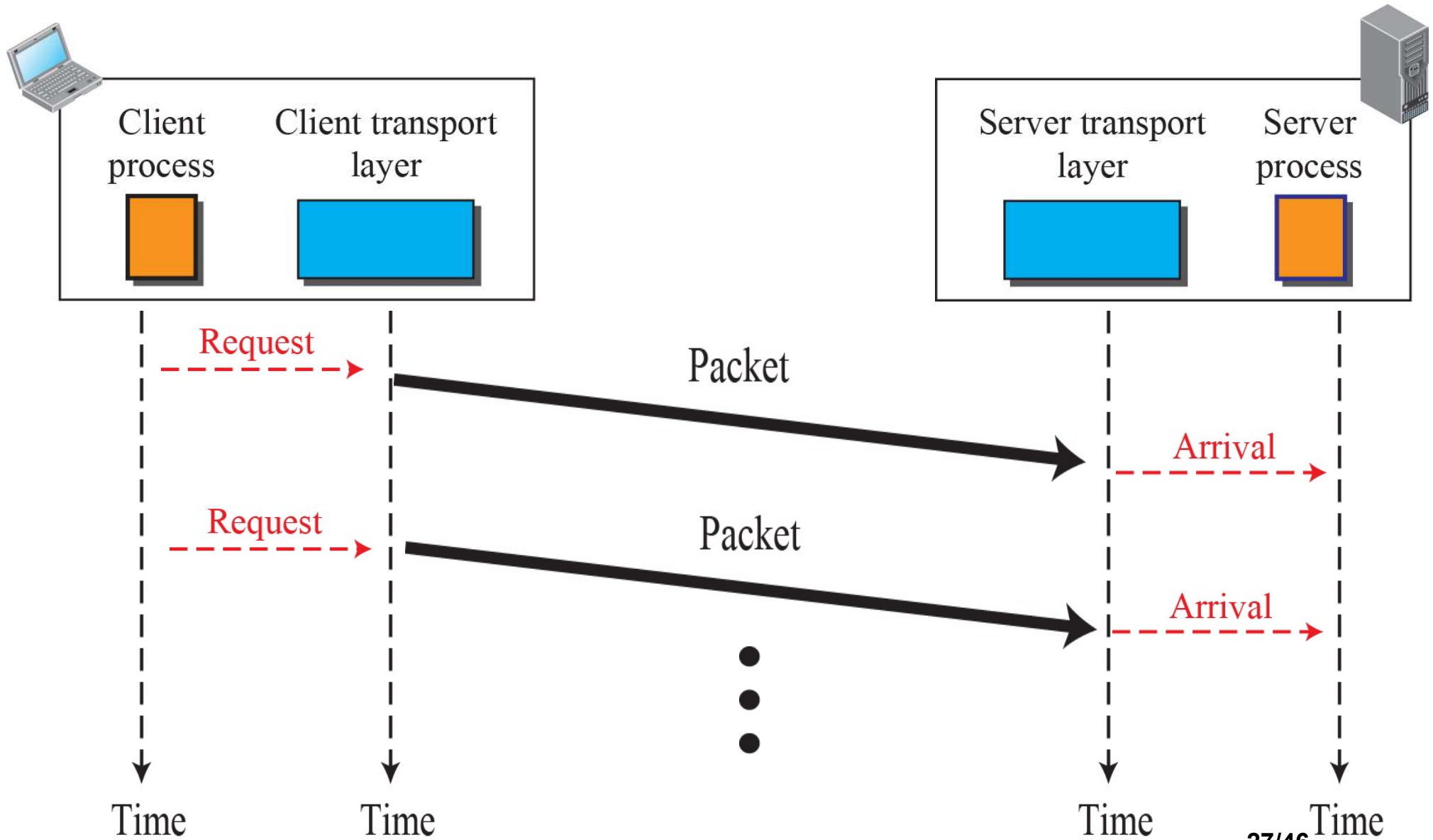
Simple protocol

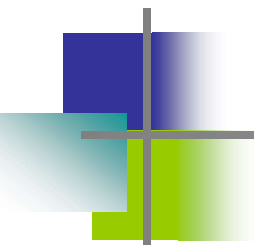


FSMs for the simple protocol



Flow diagram

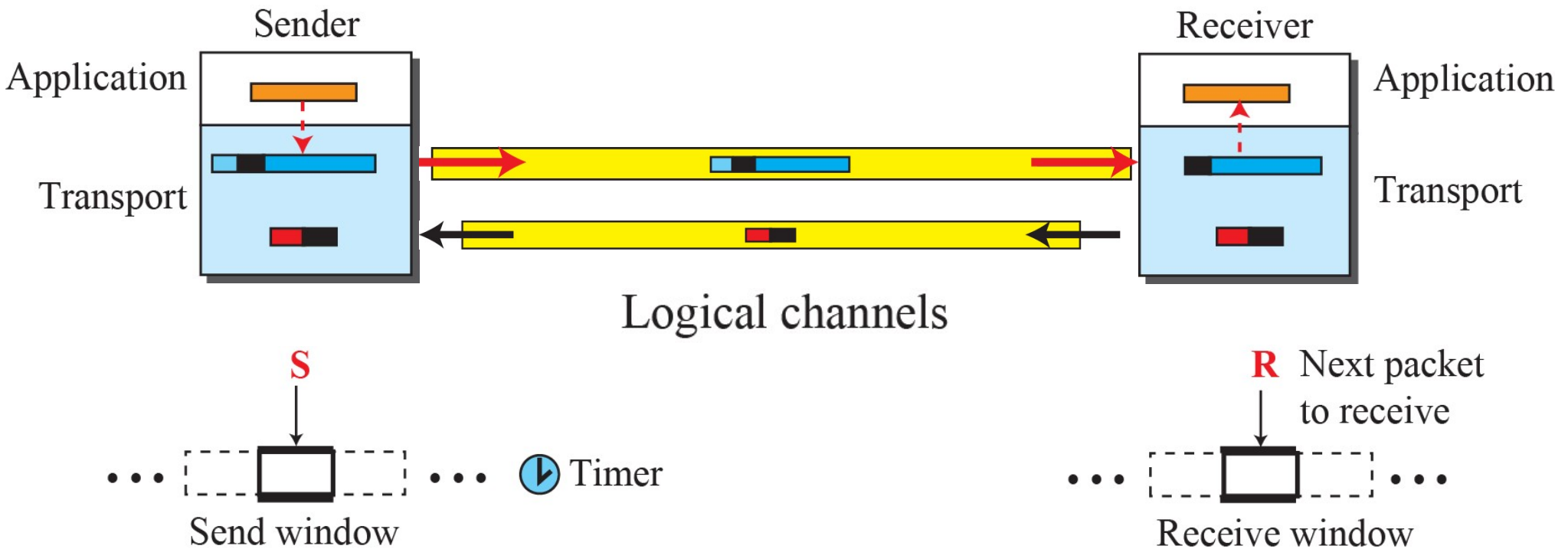




Stop-and-Wait Protocol

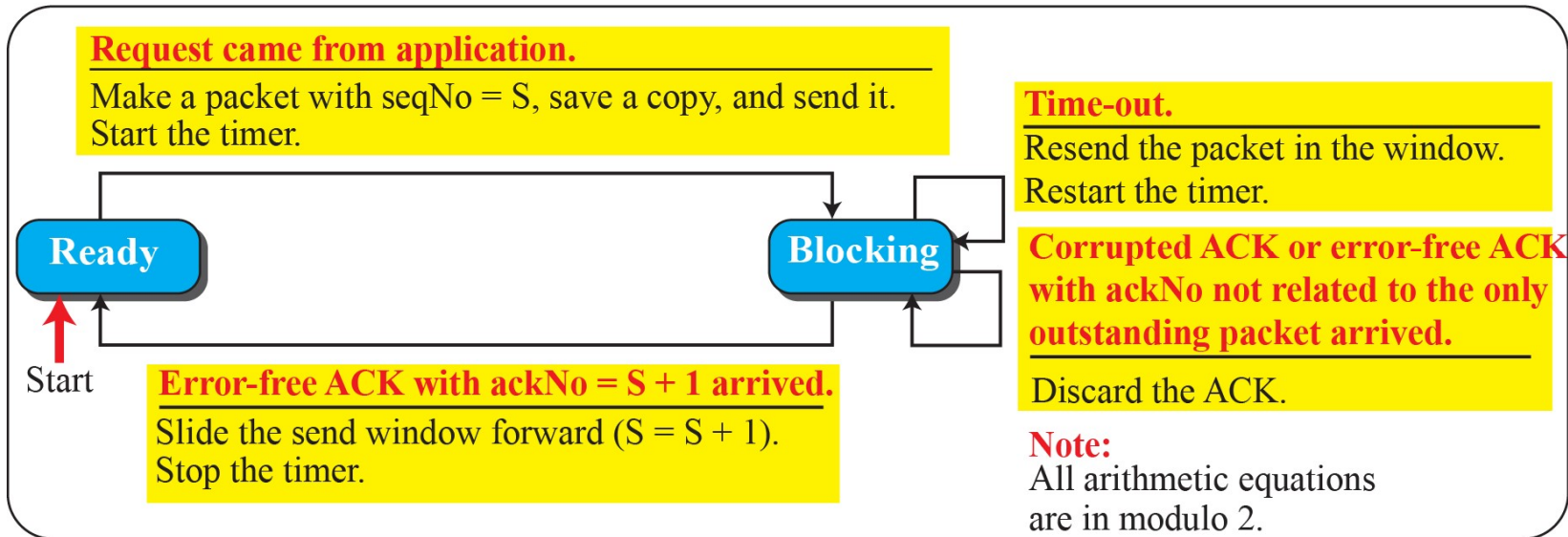
- Our second protocol is a connection-oriented protocol called the Stop-and-Wait protocol, which uses both flow and error control.
- Both the sender and the receiver use a sliding window of size 23.
- The sender sends one packet at a time and waits for an acknowledgment before sending the next one.
- To detect corrupted packets, we need to add a checksum to each data packet.
- When a packet arrives at the receiver site, it is checked. If its checksum is incorrect, the packet is corrupted and silently discarded.

Stop-and-Wait protocol

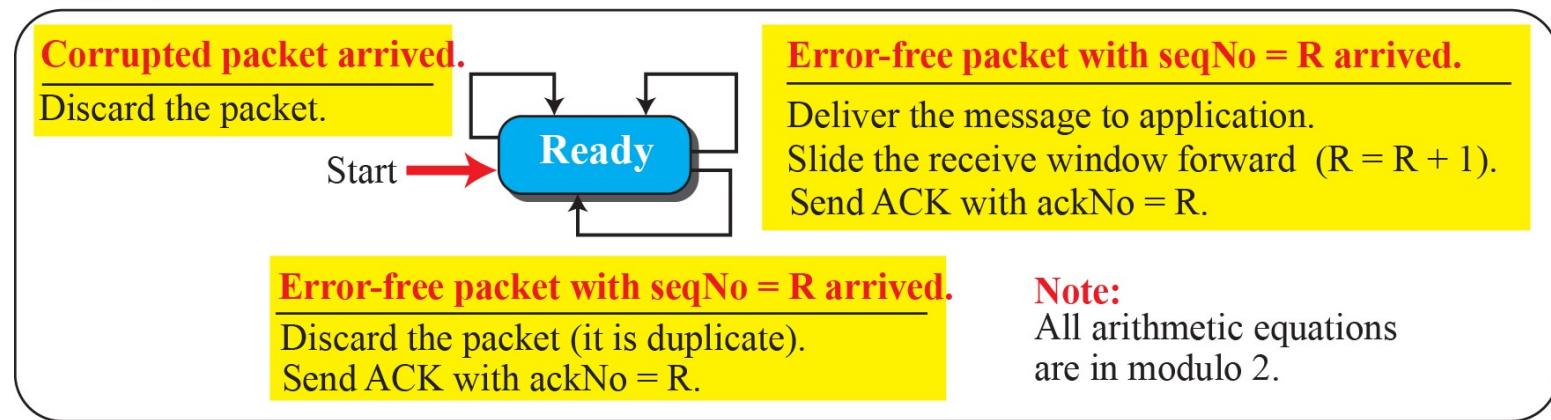


FSM for the Stop-and-Wait protocol

Sender



Receiver

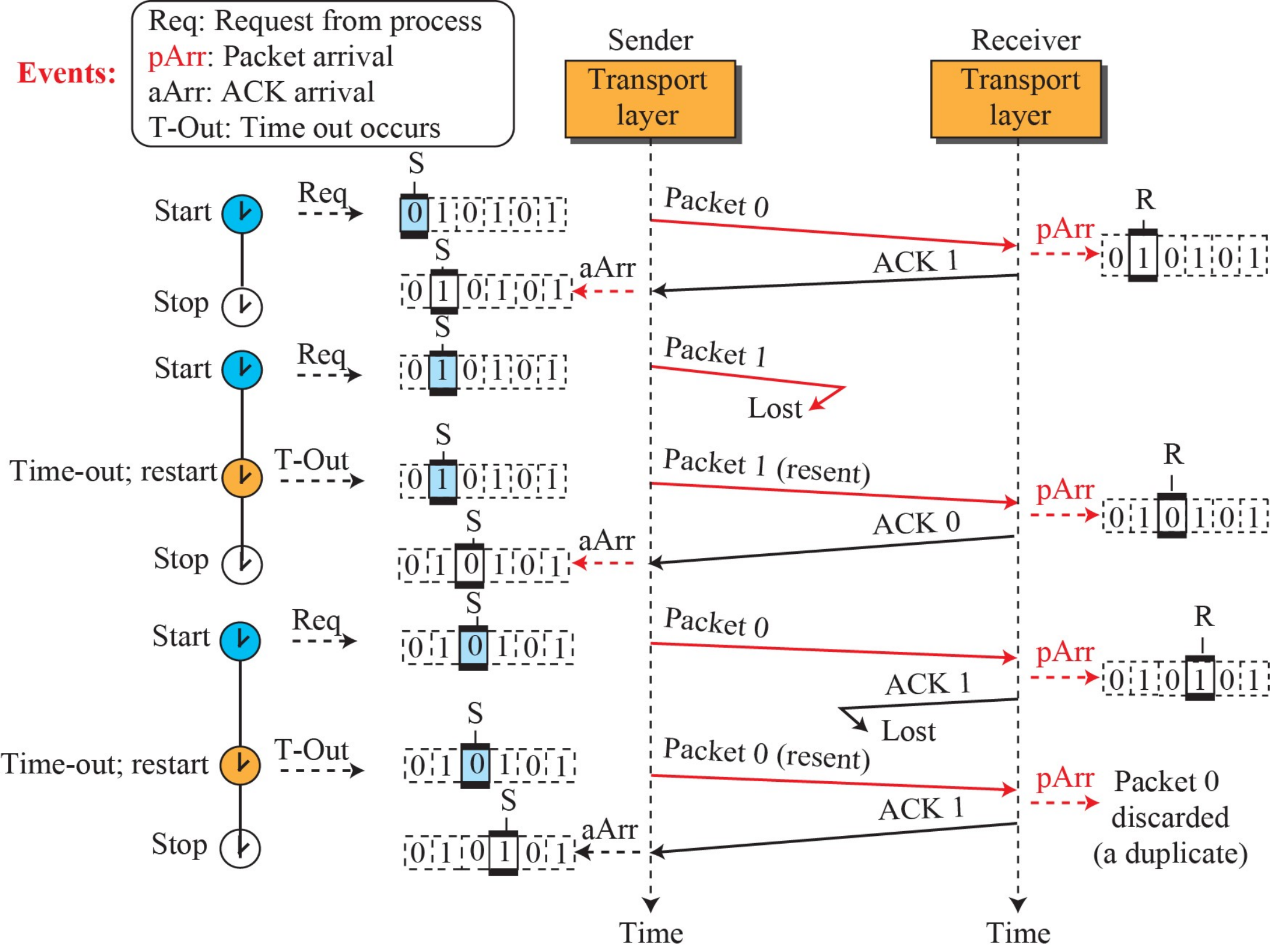


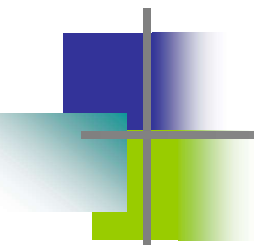
Example

Figure shows an example of the Stop-and-Wait protocol. Packet 0 is sent and acknowledged. Packet 1 is lost and resent after the time-out. The resent packet 1 is acknowledged and the timer stops. Packet 0 is sent and acknowledged, but the acknowledgment is lost. The sender has no idea if the packet or the acknowledgment is lost, so after the time-out, it resends packet 0, which is acknowledged.

Events:

Req: Request from process
pArr: Packet arrival
aArr: ACK arrival
T-Out: Time out occurs



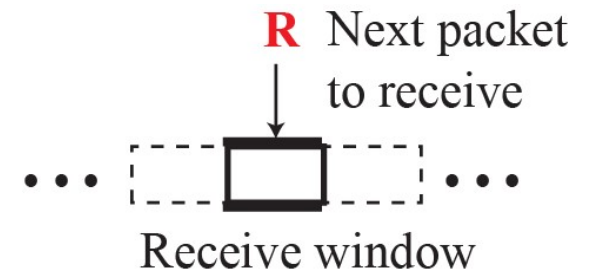
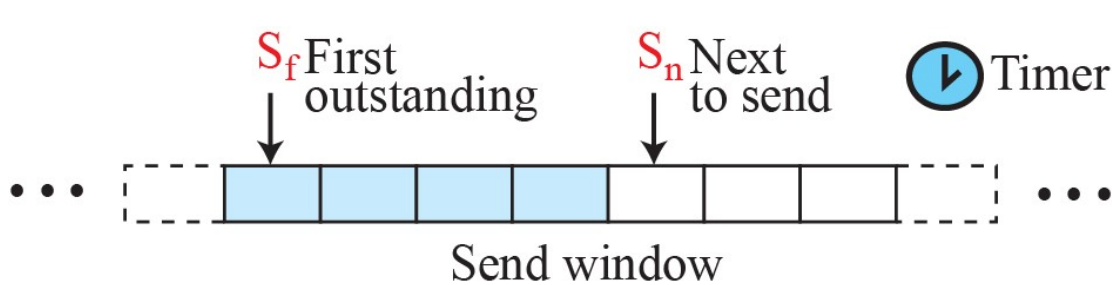
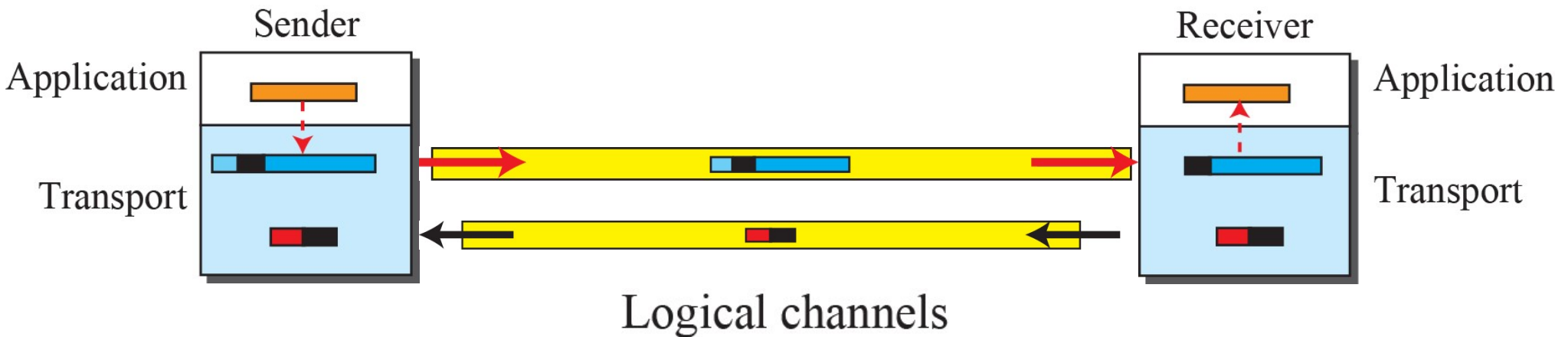
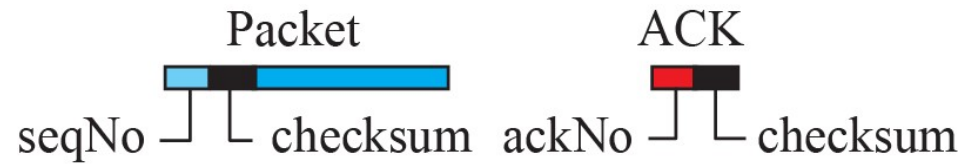


Go-Back-N Protocol (GBN)

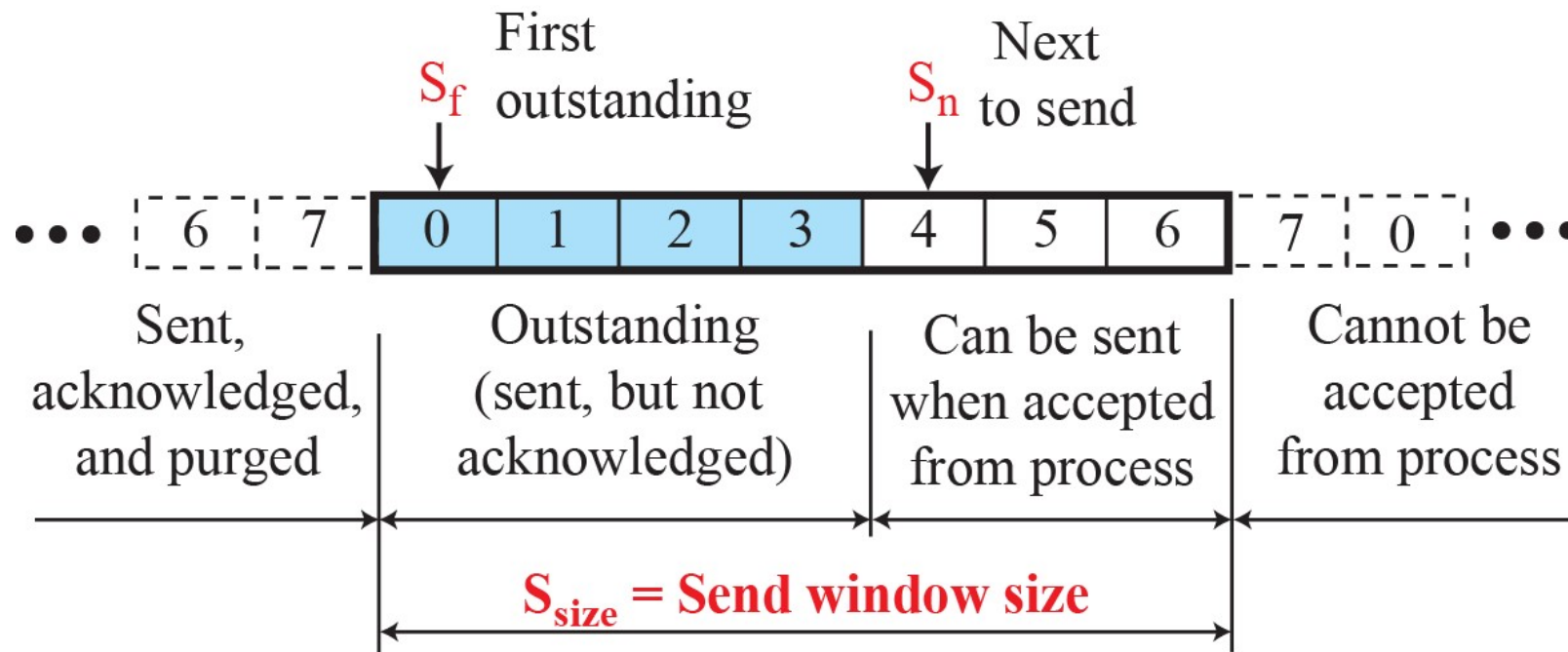
To improve the efficiency of transmission (to fill the pipe), multiple packets must be in transition while the sender is waiting for acknowledgment.

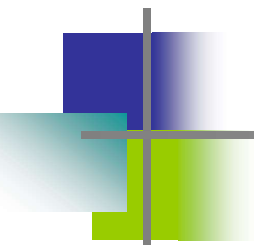
In other words, we need to let more than one packet be outstanding to keep the channel busy while the sender is waiting for acknowledgment using Go-Back-N (GBN).

Go-Back-N protocol



Send window for Go-Back-N

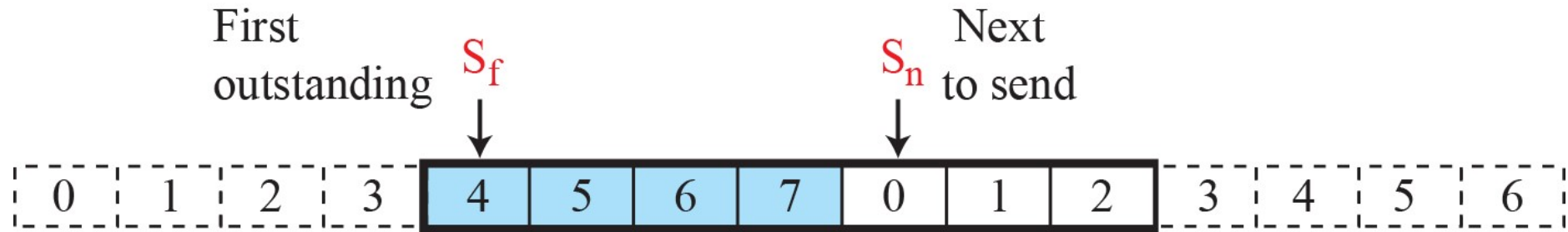




Note

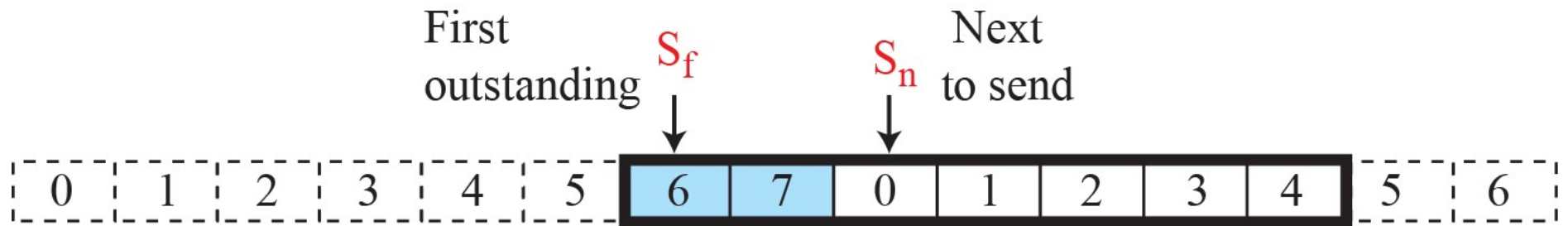
The send window is an abstract concept defining an imaginary box of size $2^m - 1$ with three variables: S_f , S_n , and S_{size} .

Sliding the send window



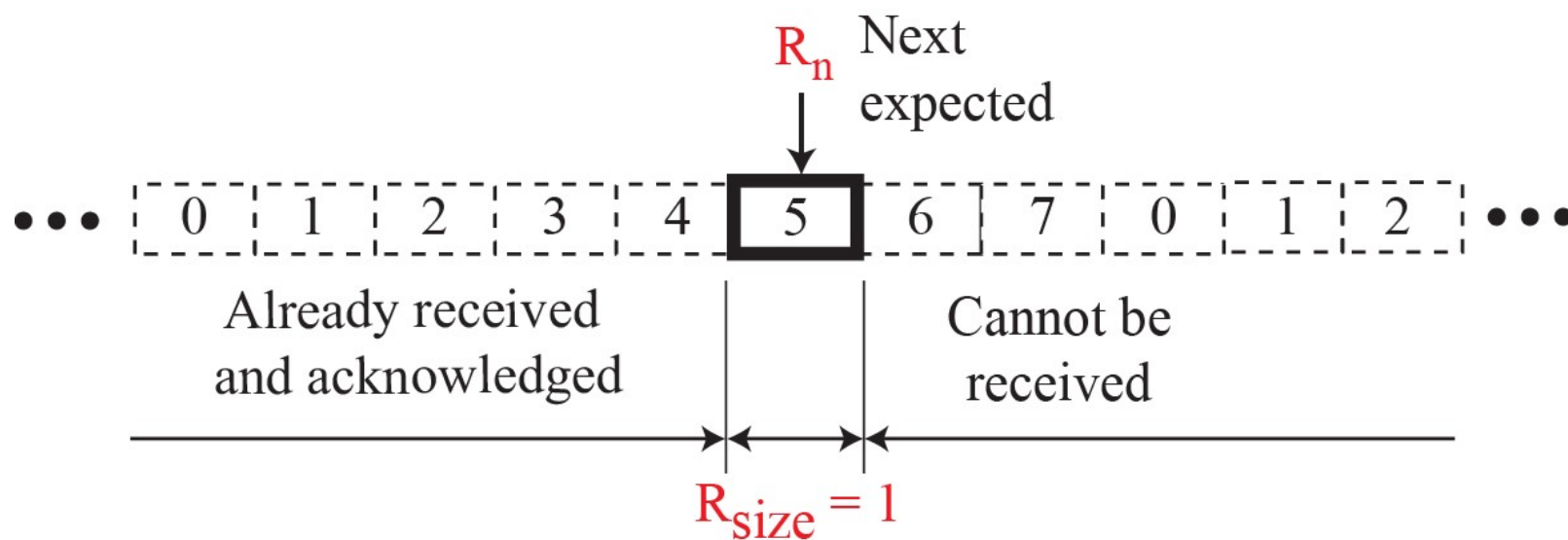
a. Window before sliding

 Sliding direction



b. Window after sliding (an ACK with ackNo = 6 has arrived)

Receive window for Go-Back-N



FSMs for the Go-Back-N protocol

Sender

Note:

All arithmetic equations are in modulo 2^m .

Time-out.

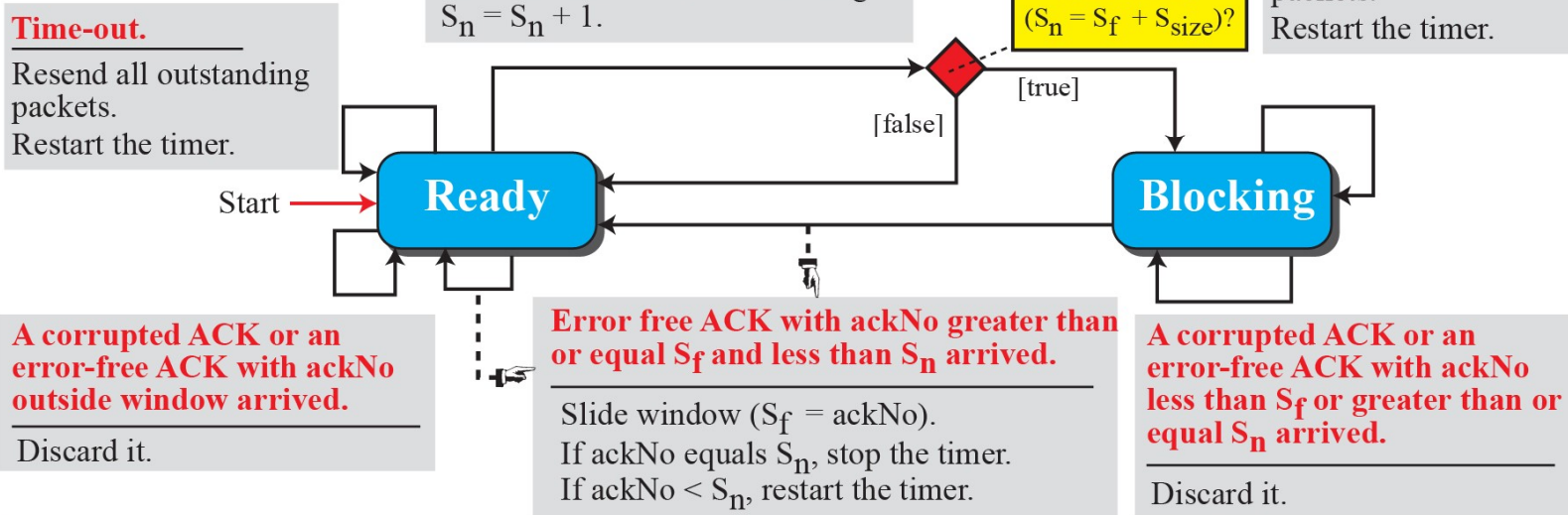
Resend all outstanding packets.
Restart the timer.

Request from process came.

Make a packet ($\text{seqNo} = S_n$).
Store a copy and send the packet.
Start the timer if it is not running.
 $S_n = S_n + 1$.

Time-out.

Resend all outstanding packets.
Restart the timer.



Receiver

Note:

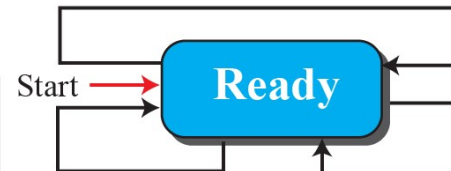
All arithmetic equations are in modulo 2^m .

Error-free packet with seqNo = R_n arrived.

Deliver message.
Slide window ($R_n = R_n + 1$).
Send ACK ($\text{ackNo} = R_n$).

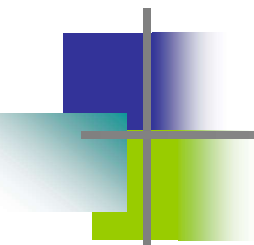
Corrupted packet arrived.

Discard packet.



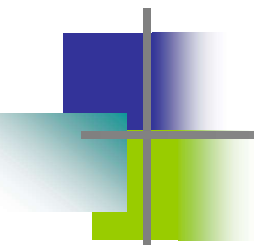
Error-free packet with seqNo = R_n arrived.

Discard packet.
Send an ACK ($\text{ackNo} = R_n$).



Note

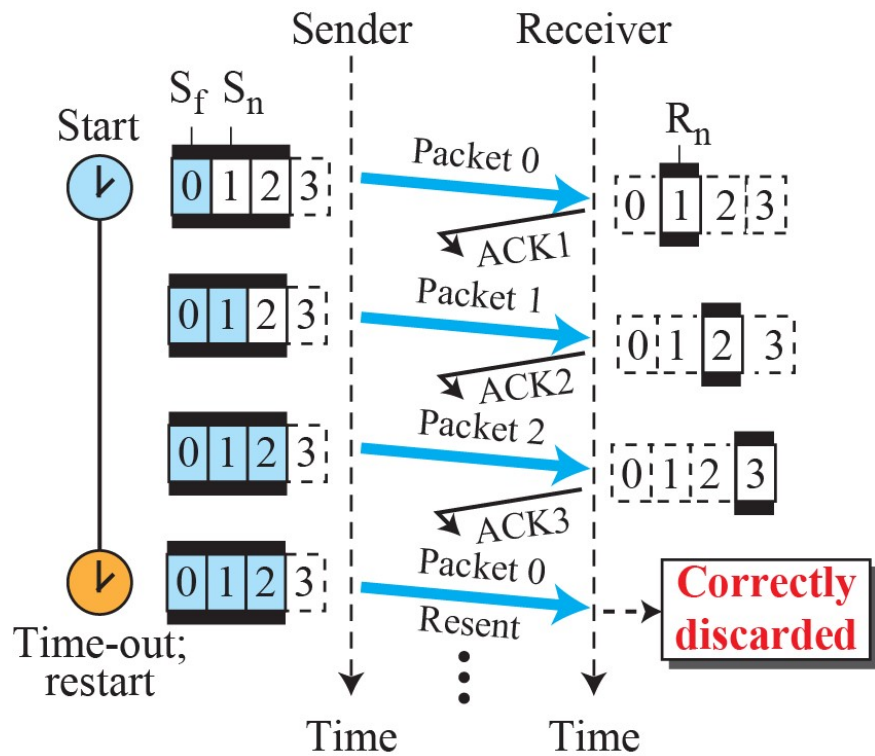
In Go-Back-N, the size of the send window must be less than 2^m ; the size of the receiver window is always 1.



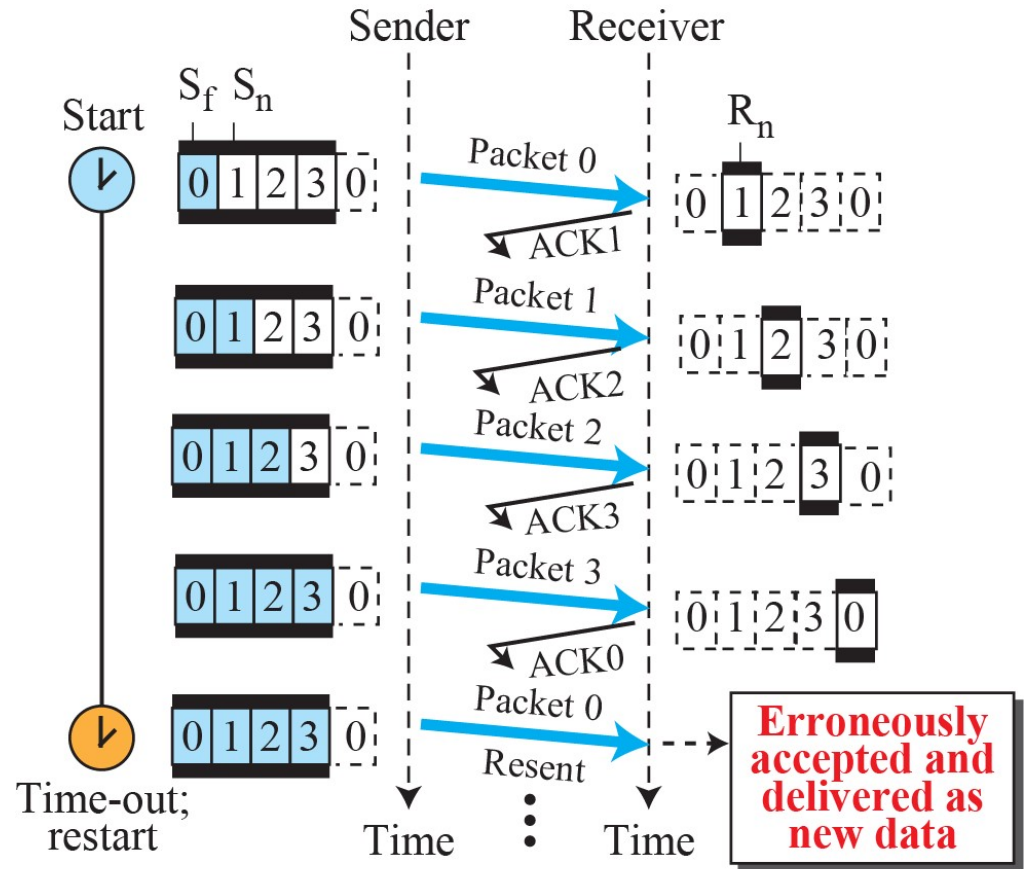
Note

Stop-and-Wait is a special case of Go-Back-N ARQ in which the size of the send window is 1.

Send window size for Go-Back-N



a. Send window of size $< 2^m$

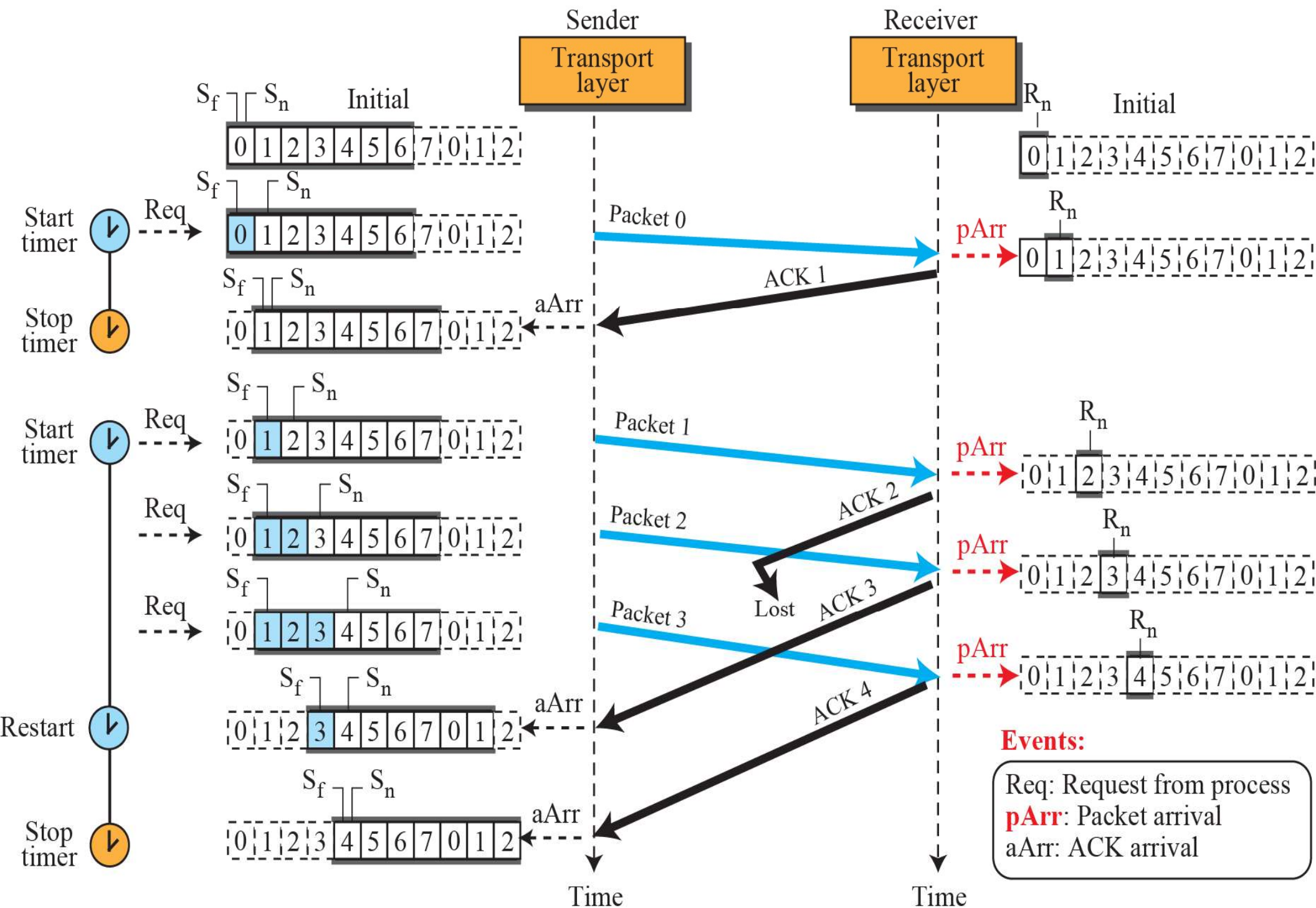


b. Send window of size $= 2^m$

Example

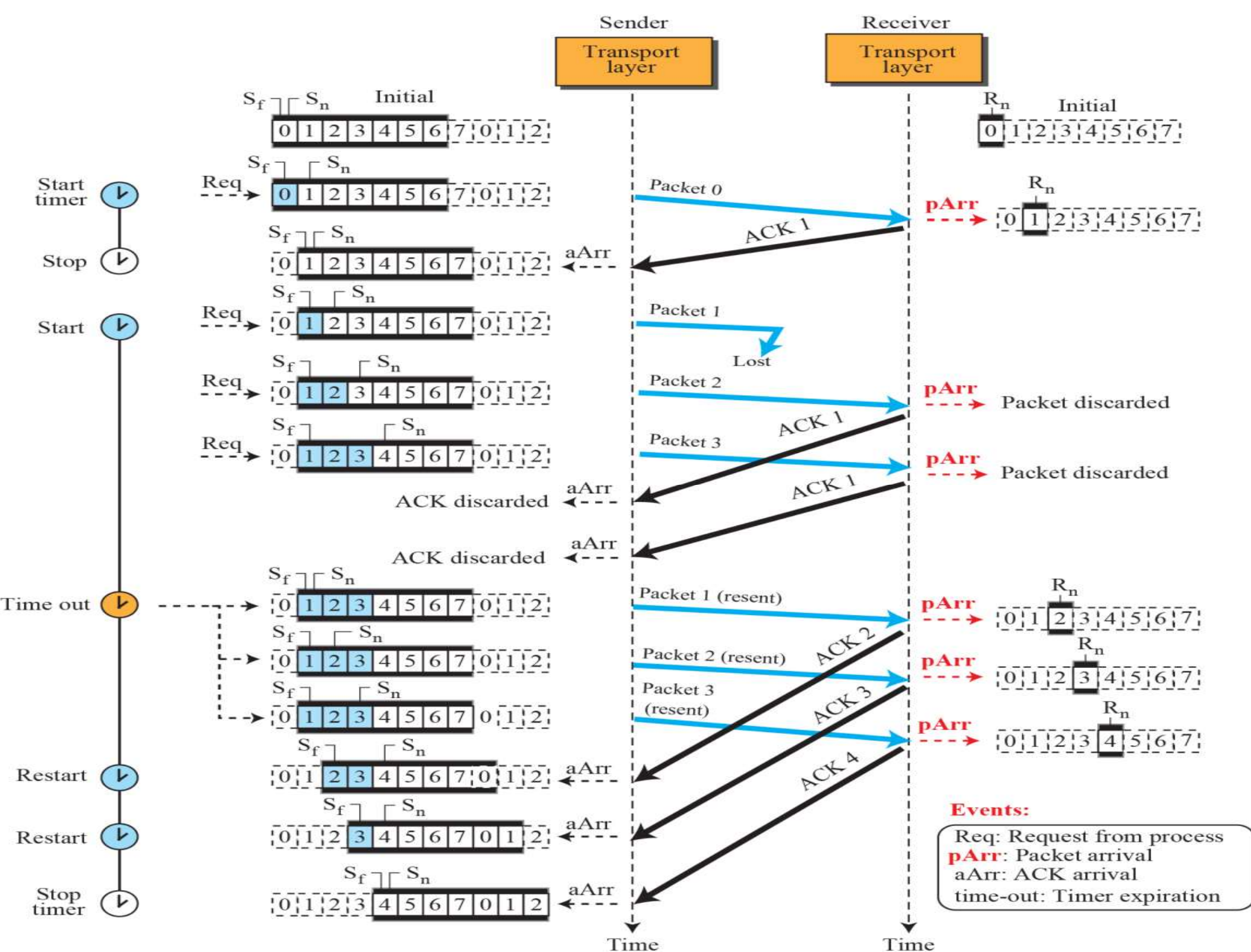
Figure 23.29 shows an example of Go-Back-N. This is an example of a case where the forward channel is reliable, but the reverse is not. No data packets are lost, but some ACKs are delayed and one is lost. The example also shows how cumulative acknowledgments can help if acknowledgments are delayed or lost.

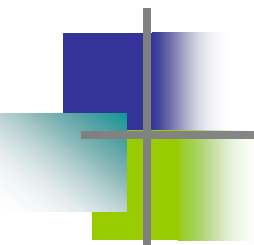
Flow diagram



Example

Figure 23.30 shows what happens when a packet is lost. Packets 0, 1, 2, and 3 are sent. However, packet 1 is lost. The receiver receives packets 2 and 3, but they are discarded because they are received out of order (packet 1 is expected). When the receiver receives packets 2 and 3, it sends ACK1 to show that it expects to receive packet 23. However, these ACKs are not useful for the sender because the ackNo is equal to S_f , not greater than S_f . So the sender discards them. When the time-out occurs, the sender resends packets 1, 2, and 3, which are acknowledged.

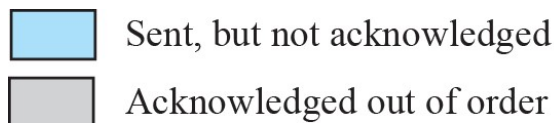
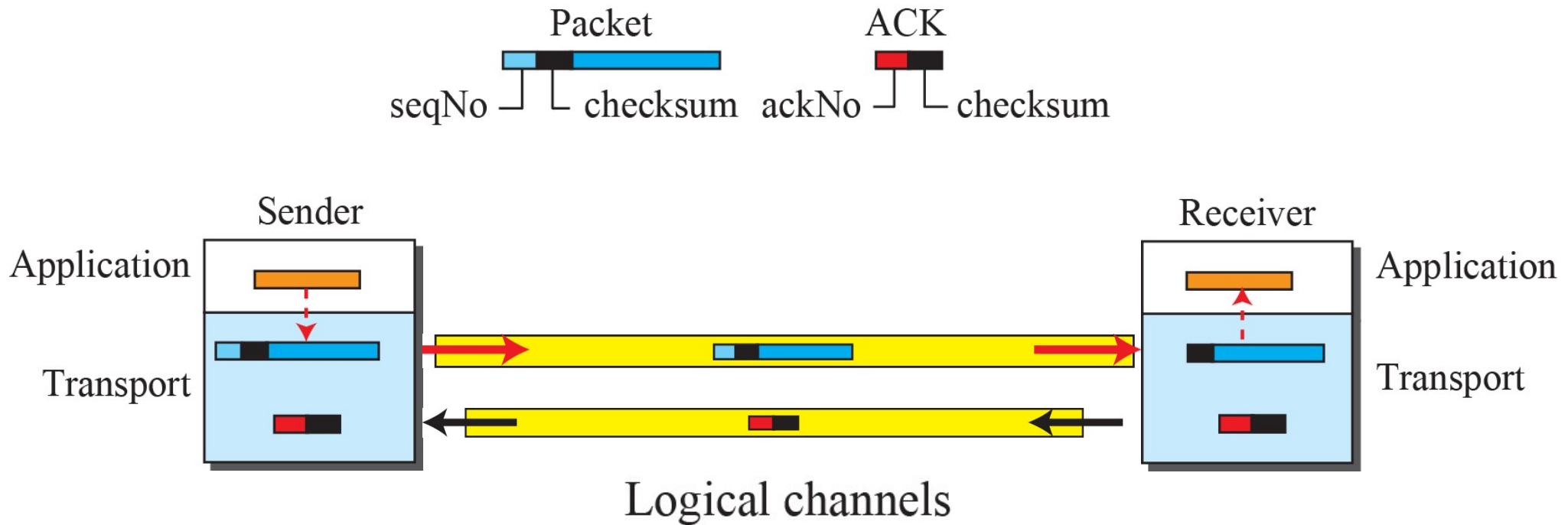




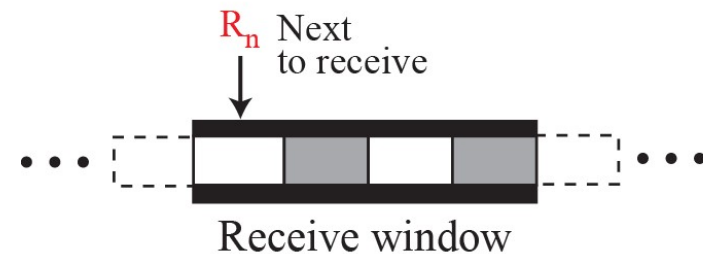
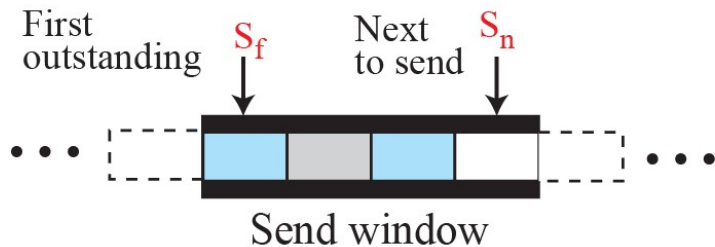
Selective-Repeat Protocol

The Go-Back-N protocol simplifies the process at the receiver. The receiver keeps track of only one variable, and there is no need to buffer out-of-order packets; they are simply discarded. However, this protocol is inefficient if the underlying network protocol loses a lot of packets. Each time a single packet is lost or corrupted, the sender resends all outstanding packets, even though some of these packets may have been received safe and sound but out of order.

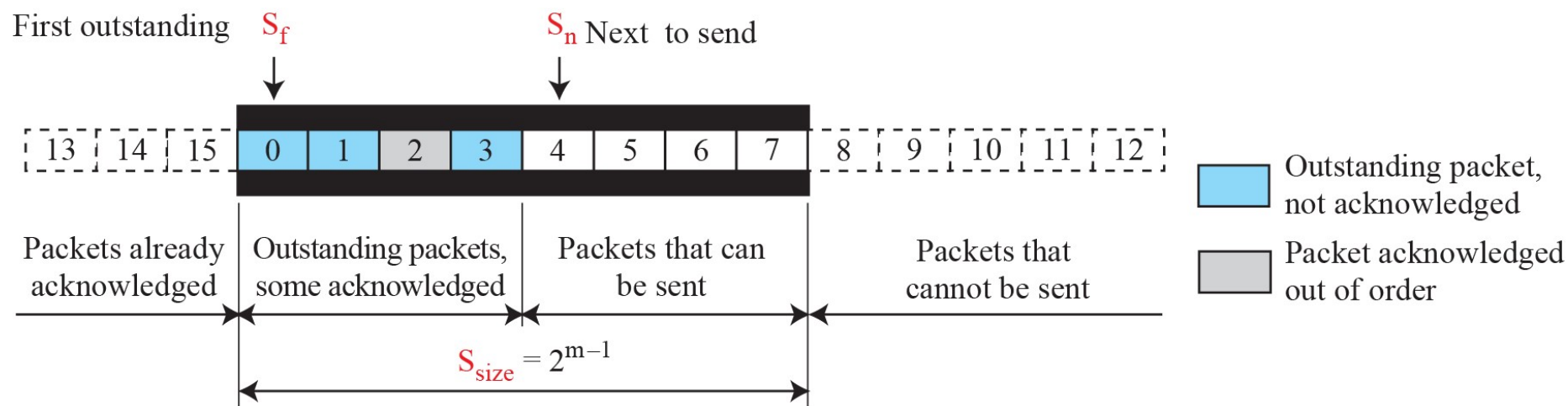
Outline of Selective-Repeat



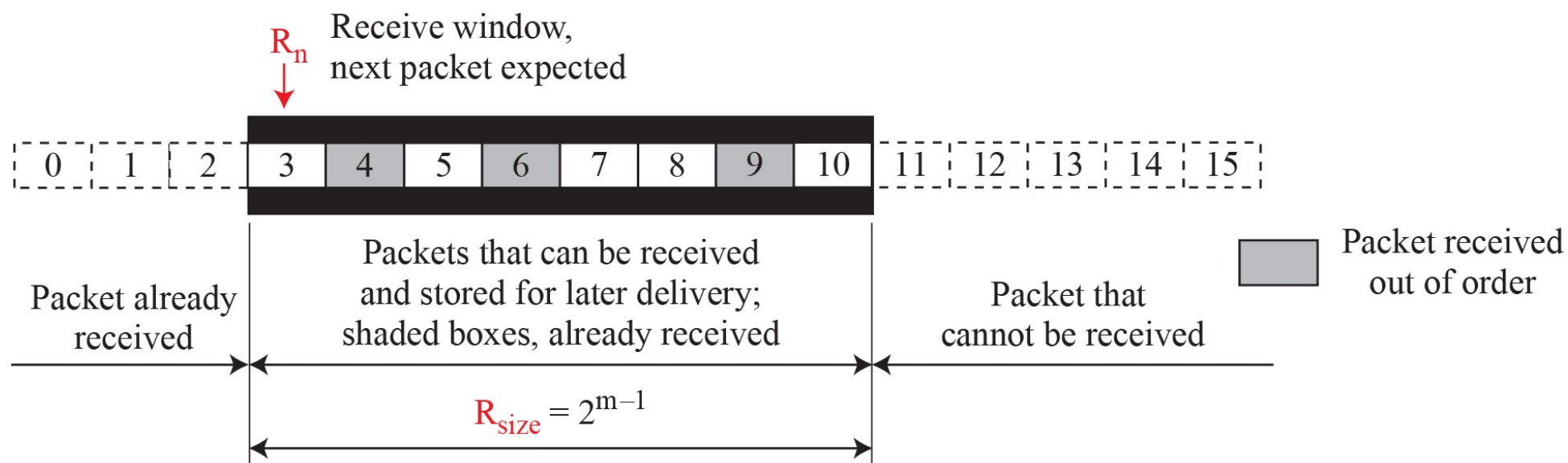
Timer



Send window for Selective-Repeat protocol

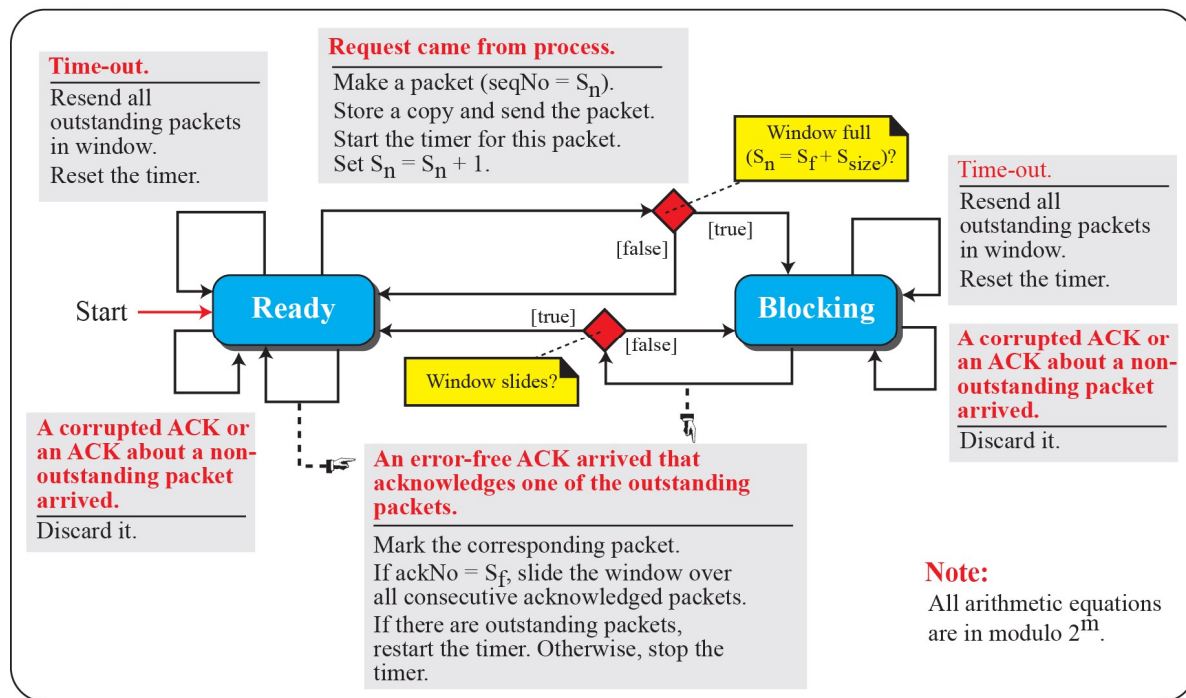


Receive window for Selective-Repeat protocol

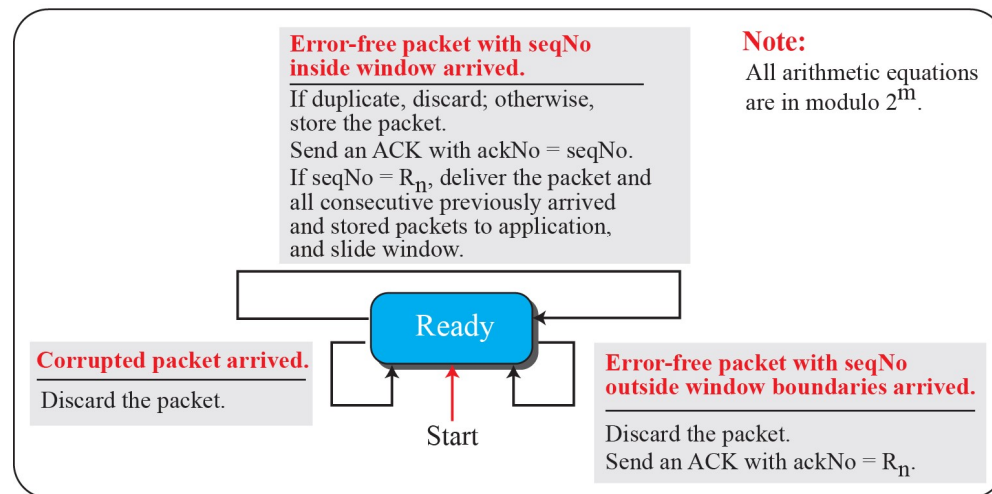


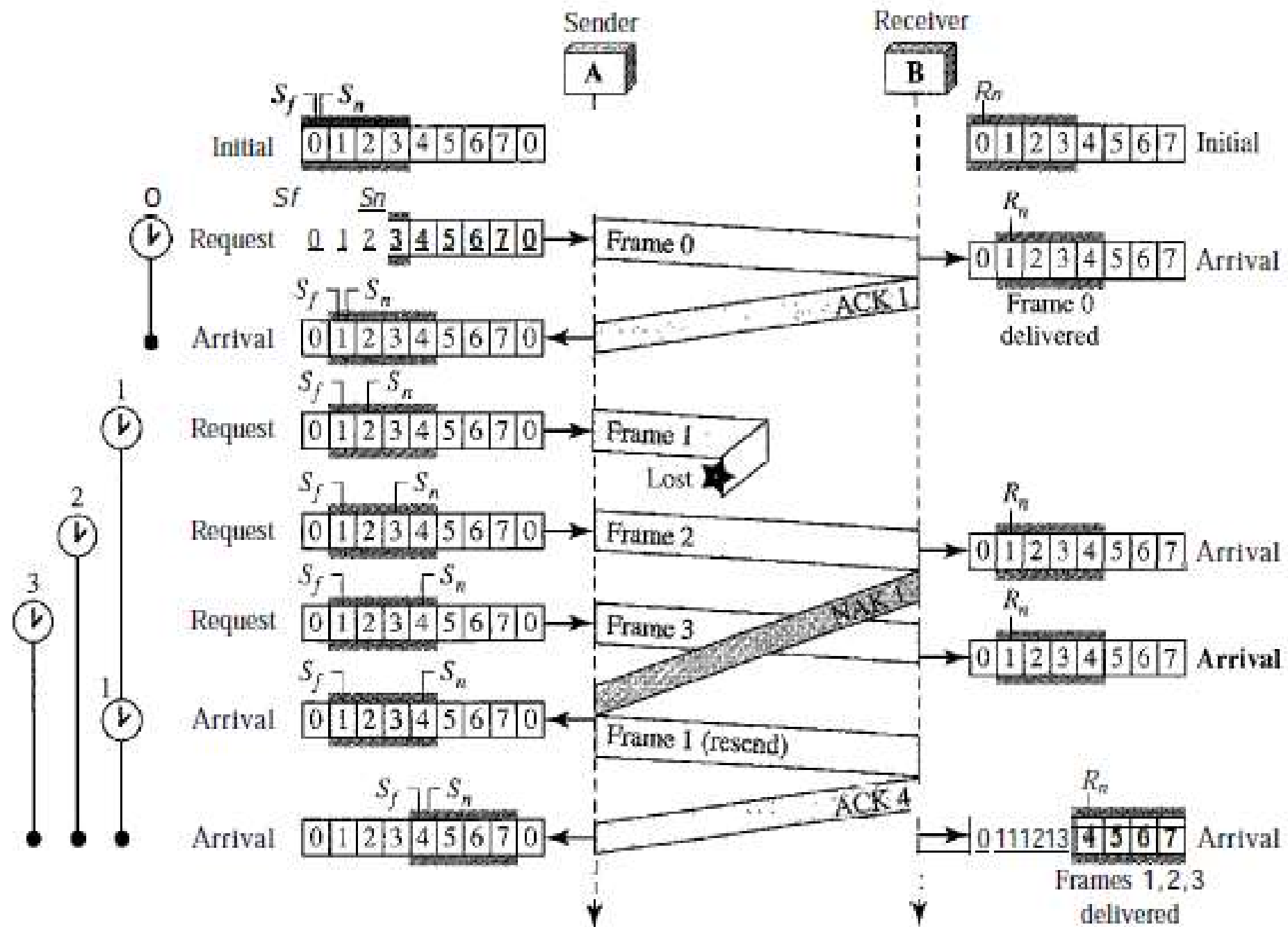
FSMs for SR protocol

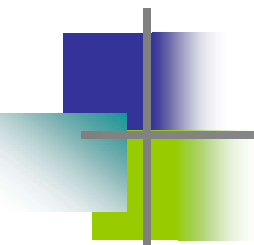
Sender



Receiver



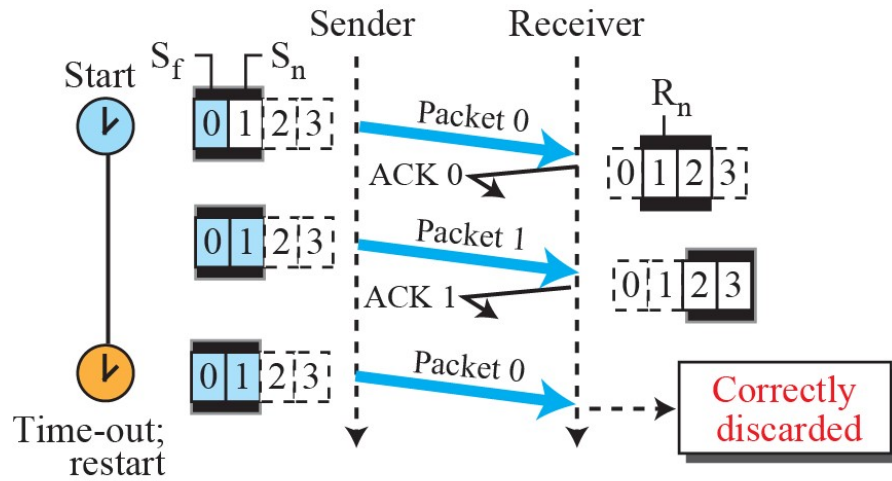




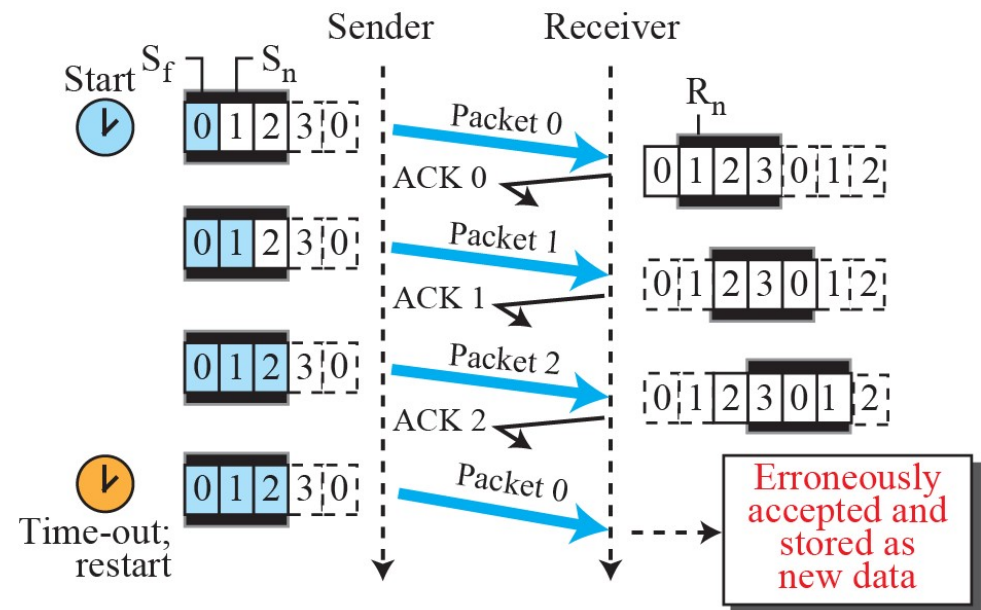
Note

In Selective Repeat, the size of the sender and receiver window must be at most one-half of 2^m .

Selective-Repeat, window size



a. Send and receive windows of size $= 2^m - 1$



b. Send and receive windows of size $> 2^m - 1$

Check Your Understanding

What is the difference between Go back N and Selective Repeat Protocol?

Outcome

Student should be able to understand

- Transport Layer Flow control protocols
 - Simple protocol
 - Stop and wait protocol
 - Go back N protocol
 - Selective Repeat protocol

THANK YOU